

Academic Thesis for the Attainment of the Degree of
Master of Science
at the Technical University of Munich

An Algorithmic Framework for Very Large-Scale Capacitated Vehicle Routing Problems

Examiner: Prof. Dr. Stefan Minner
Chair of Logistics and Supply Chain Management
Technische Universität München

Supervisor: Till Krieger, M.Sc.

Degree Program: Management & Technology Master

Submitted by:
Lucca Altendeitering
Stolzestr. 21, 44789 Bochum
Tel.: +49 162 7630077
Matriculation Number: 3782307

Date of Submission: 30.09.2026

An Abstract should not include more than 150 words.

Table of Contents

List of Figures	iv
List of Tables	v
Abbreviations	vi
1 Introduction	1
2 Literature Review	2
2.1 State-of-the-Art Solvers for Large-Scale CVRP	2
2.2 Scalability Strategies	2
2.2.1 Parallelization	2
2.2.2 Set Covering and Set Partitioning	2
2.2.3 Decomposition	3
2.2.4 Machine Learning	3
2.3 Algorithmic Frameworks	3
2.4 Remaining Research Gap	3
3 Problem Statement	4
4 Framework	5
4.1 Hierarchical Adaptive Operator Selection	7
4.2 Decomposition	11
4.3 Routing	12
4.4 Boundary-Guided Adaptive Iterated Local Search	13
4.5 Route Pool and Set Partitioning	18
4.6 Improvement	22
4.7 Pipeline Orchestration	22
5 Computational Study	27
5.1 Experimental Design	27
5.1.1 Framework Implementation	27
5.1.2 Dataset	28
5.1.3 Repetitions	28
5.1.4 Hyperparameters	30
5.2 Results	31
5.2.1 Performance Comparison against State-of-the-Art Mono-lithic Solvers	31

5.2.2	Performance by Instance Characteristics	36
5.2.3	HAOS Weights	37
5.2.4	Convergence Behavior	38
5.3	Limitations	38
6	Conclusion	39
6.1	Highlights	39
6.2	Future Research	39
	Bibliography	40
	Appendix	45
1	Set Covering Formulation	45
2	XL Instance Detailed Results	46

List of Figures

4.1 Sequential control flow of one DRISPI run: the DRI loop, the route pool, and periodic SC/SP recombination.	6
4.2 Core allocation and cross-iteration overlap in DRISPI's parallelized pipeline.	24
5.1 Empirical CDF of the mean-of- N gap to the current BKS. KGLSXXL, LKH-3, and OR-Tools are omitted (Section 5.2.1): their gaps would compress the visible range for the other six.	34
5.2 Mean-of-3 gap to the current Best Known Solution (BKS) by instance characteristic, across all 100 XL instances (error bars are one sample standard deviation).	37

List of Tables

4.1 Default Immediate and Deferred HAOS Rewards.	9
4.2 Inputs Used By Each Decomposition Method.	12
4.3 BG-AILS Parameters and Their Defaults.	17
5.1 Forecast standard error and 95% half-width of $\bar{G}$ by seed count S	29
5.2 Mean percentage gap to the current BKS, DRISPI against eight published solvers, sorted by ascending mean-of- N gap over all instances.	32
5.3 Instances with the largest mean-of-3 gap to the current BKS (codes per Section 5.1.2).	35
5.4 Instances where DRISPI's best-of-3 cost matches or beats a published reference.	35
6.1 Detailed results for DRISPI on the 100 XL instances.	46

Abbreviations

BKS Best Known Solution.

CVRP Capacitated Vehicle Routing Problem.

1 Introduction

Motivation: Whats the real world problem and application, why is it important? Why are we doing this research now? Mention the BKS challenge and the project study as underlying motivators for this thesis.

Status Quo: What is the current state of the art? What are the limitations of the current state of the art? What are the challenges? Mention np-hardness and the issues that come with monolithic approaches at scale.

Research Questions: What is the focus of this thesis? Name primary and secondary research question and outline how they will be answered/evaluated.

Contributions: Specify the concrete contributions made by this thesis. This should include the general approach, and specifically the novel contributions of HAOS, BG-AILS, and the parallelized DRISPI framework approach.

Structure: Overview of the chapters and their content.

2 Literature Review

Cite papers with `\cite{BibTeX key}`

There is a comprehensive introduction to Operations Research by Winston (2007). It may be beneficial to consult comprehensive review articles within the relevant research domains (Dekker et al., 2012; Matl et al., 2018; Zhang and van Woensel, 2023).

Also use the introduction to this chapter to outline the general field of VRP and related problems. Explain np-hardness, the background VRP and CVRP, and how research has been progressing in the field.

2.1 State-of-the-Art Solvers for Large-Scale CVRP

Namedrop the big monolithic solvers for CVRP and their limitations. Especially mention the four solvers which are used in DRISPI: AILS-II, FILO, FILO2, and HGS-CVRP.

The main limitation to be mentioned here is the computational complexity and how it scales superexponentially with size of the problem.

Also explain the general approaches of these monoliths and what else is out there.

2.2 Scalability Strategies

Write here about different approaches to scale when it comes VRPs. This should include parallelization (-> prepares for DRISPI), decomposition or divide and conquer approaches (-> prepares for DRI), and maybe some recent machine learning approaches (-> prepares for HAOS).

2.2.1 Parallelization

xyz

2.2.2 Set Covering and Set Partitioning

xyz

2.2.3 Decomposition

xyz

2.2.4 Machine Learning

xyz

2.3 Algorithmic Frameworks

Idk about this section, but this would be a great way to introduce the approaches of Christoph (DRI for CCRPTW and DRSCI for HVRP). Maybe there are other approaches like this which include different strategies.

I need to be clear here what the difference between a framework, solver, and heuristic is. But if done well, this is a very nice way to introduce these terms and concepts and a good bridge to the research gap.

2.4 Remaining Research Gap

This should seamlessly connect to the research questions and pick them back up. It should present the state of the art approaches as unfit for the future of larger and larger VRPs.

Also, it should present that a combination of different strategies to tackle scale is unexplored and that the release of this new problem set is the perfect opportunity to explore this.

This section should get a sick figure/table to illustrate the research gap and how DRISPI combines all these scalability strategies which have been explored only in isolation so far.

3 Problem Statement

As introduced in Chapter 1, this work addresses the Capacitated Vehicle Routing Problem (CVRP) at a very-large scale. We now state the problem formally, discuss its computational complexity, and define the scope of the computational study.

The CVRP is defined on a complete undirected graph $G = (V, E)$, where $V = \{0\} \cup C$ is the set of locations. Node 0 represents the depot, and $C = \{1, \dots, n\}$ is the set of customers to be served, so that $n = |C|$. Each edge $\{i, j\} \in E$ carries a non-negative travel cost $d_{ij} \geq 0$, and each customer $i \in C$ has an associated demand $q_i > 0$. A homogeneous fleet of vehicles, each with a maximum capacity Q , is dispatched from and returns to the depot. The objective is to find a set of routes, each starting and ending at the depot, such that every customer is visited exactly once, no route exceeds the vehicle capacity, and the total travel cost is minimized.

The CVRP is NP-hard (Lenstra and Kan, 1981), a direct consequence of its generalization of the Traveling Salesman Problem. In practical terms, the number of feasible solutions grows factorially with n , effectively rendering exact methods intractable beyond a few hundred customers. While branch-cut-and-price algorithms can solve small and medium instances to proven optimality () **[TODO: CITE: branch-cut-and-price exact method]**, they are not viable at the scales considered here. This necessitates heuristic and metaheuristic approaches, which trade optimality guarantees for high-quality solutions within practical time budgets. Even within the heuristic paradigm, as Chapter 2 discusses, the cost of operating on the full problem simultaneously grows rapidly with n : neighborhood evaluation, memory requirements, and local search effectiveness all deteriorate at very-large scale, motivating the decomposition-based approach taken in this work.

This work focuses specifically on the XL instances of the CVRPLib benchmark set (Queiroga et al., 2026), containing between 1,000 and 10,000 customers. These instances span a wide range of structural characteristics, including depot positioning, customer distribution, demand heterogeneity, and average route length, making them a demanding and representative testing set for large-scale CVRP solvers. The DRISPI framework, proposed in Chapter 4, is designed specifically to operate at this scale by decomposing the full instance into tractable subproblems, which are then solved independently and parallelized using existing state-of-the-art solvers.

4 Framework

This chapter presents DRISPI, the framework this thesis develops for the CVRP at very-large scale (Chapter 3). DRISPI repeatedly executes a decompose-route-improve (DRI) loop. Each iteration first partitions the customer set into smaller subclusters and solves one routing subproblem per subcluster with an existing monolithic solver, then concatenates the resulting routes and revises them near the subcluster boundaries with a dedicated improvement stage, boundary-guided adaptive iterated local search (BG-AILS). Hierarchical adaptive operator selection (HAOS) draws the decomposition and routing configuration for every iteration from a set of weighted alternatives and updates those weights from the outcomes it observes, rather than fixing one configuration for the whole run. Every route the DRI loop produces enters a persistent route pool; at scheduled intervals, DRISPI steps back from the current incumbent and recombines the pool through a set covering or set partitioning (SC/SP) model, followed by one further improvement pass. The loop continues until a wall-clock time limit or a run of iterations without improvement is reached.

Figure 4.1 shows the resulting control flow for one instance. HAOS (Section 4.1) draws one configuration per iteration; decomposition (Section 4.2) partitions the customer set into k subclusters under that configuration; routing (Section 4.3) solves each subcluster independently and concatenates the routes; BG-AILS (Section 4.4) revises the routes near the subcluster boundaries. Both the routing stage and BG-AILS add their routes to the route pool, and either can become the new incumbent if it improves on the current one. At scheduled intervals, the same iteration also runs set covering or set partitioning (Section 4.5) over the pool accumulated so far, followed by one further improvement pass (Section 4.6) whose output feeds the pool again and can likewise become the new incumbent; on every other iteration, this pair of stages is skipped. The loop then either returns to a fresh HAOS draw or stops. Figure 4.1 shows this control flow as one sequential loop; Section 4.7 shows how DRISPI executes it with several stages running in parallel across iterations instead.

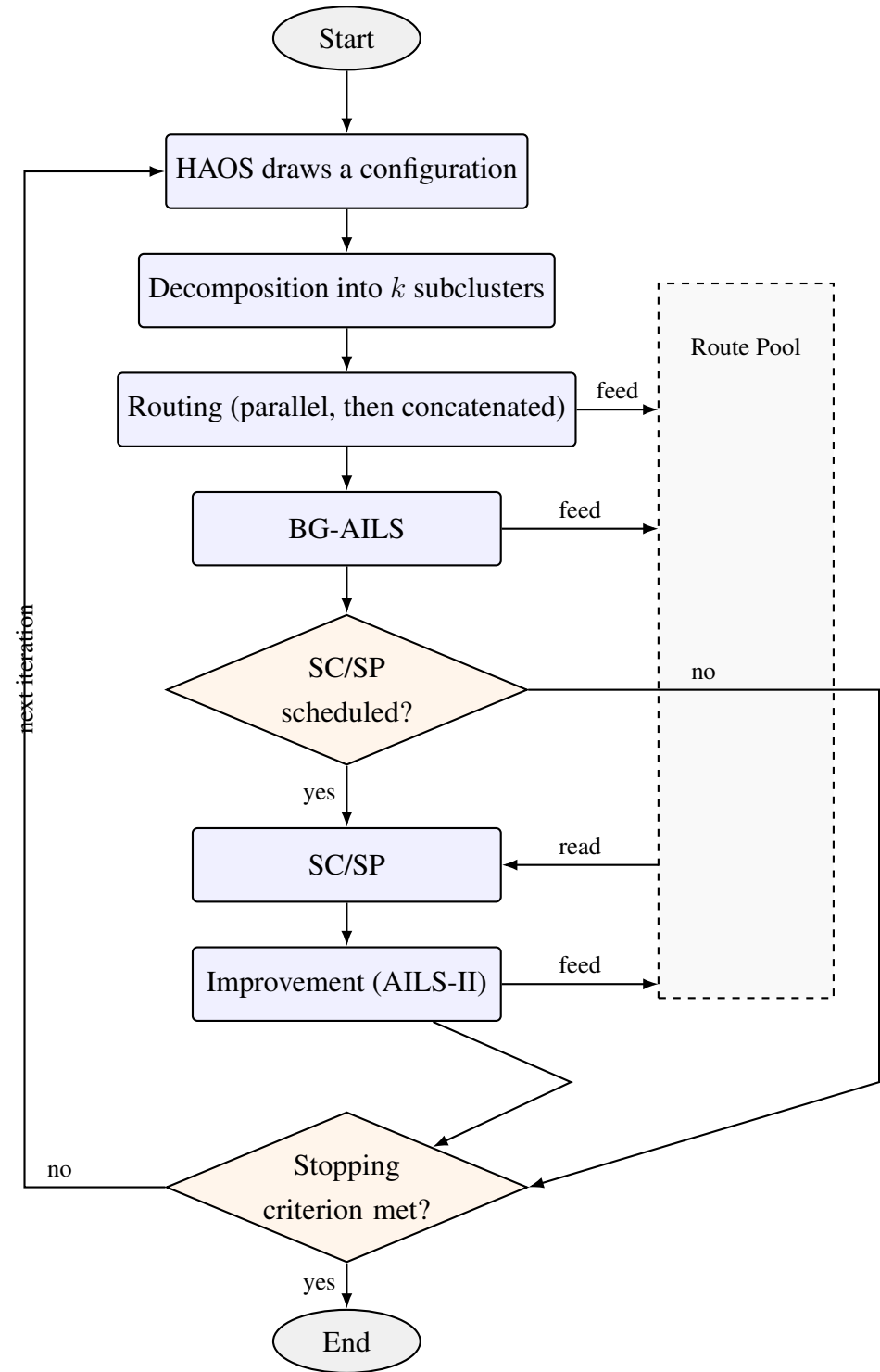


Figure 4.1: Sequential control flow of one DRISPI run: the DRI loop, the route pool, and periodic SC/SP recombination.

The remaining sections address one stage each, in the order Figure 4.1 shows: Section 4.1 on HAOS, Section 4.2 on decomposition, Section 4.3 on routing, Section 4.4 on BG-AILS, Section 4.5 on the route pool and SC/SP, and Section 4.6 on the post-SC/SP improvement pass. Section 4.7 closes the chapter by assembling these stages into the parallelized pipeline DRISPI actually runs, together with the pseudocode for its control loop.

4.1 Hierarchical Adaptive Operator Selection

Ideally, a heuristic would always be tuned to no more than one specific problem instance. That would promise both, an effective and efficient approach configured to exploit that instance’s size, spatial and demand distributions, and fleet capacity. Recent benchmark sets however have deliberately combined a wide range of these characteristics to test whether performance transfers across instance structures (Uchoa et al., 2017; Queiroga et al., 2026). That is why finding and selecting a tailored configuration for every instance requires a costly tuning campaign. That is why most benchmarks are compromising on one fixed configuration for the entire problem set. This framework aims to forego this compromise through Hierarchical Adaptive Operator Selection (HAOS), an approach to adapt DRISPI’s configuration while solving the problem instead.

The principal methodological inspiration is adaptive large neighborhood search. Competing destroy and repair operators receive weights based on observed performance and are selected by roulette wheel (Ropke and Pisinger, 2006). The same adaptive layer has been applied across five vehicle routing variants (Pisinger and Ropke, 2007). HAOS adopts this principle but moves the adaptive choice outside a single local search. It instead controls the decomposition granularity, the representation and method used for clustering, the demand contribution cluster formation, as well as the subsolver. The adaptation is online because rewards update the weights while the current instance is being solved. The weights of each operator summarize observed performance on that instance. The Convergence to a globally optimal configuration is neither required nor guaranteed.

The operator choices can also be interpreted as arms of a multi-armed bandit. After an arm is drawn, the search observes a reward but does not know what the unselected arms would have produced. This analogy connects HAOS to the exploration–exploitation problem (Auer et al., 2002) and to bandit-based adaptive operator selection (Fialho et al., 2010). However, HAOS does not implement a confidence-bound policy, an explicit exploration bonus, or a separate exploratory action. Its roulette probabilities follow the current weights directly. Stochastic sampling leaves every positive-weight option selectable, while the probability-floor transformation described below raises normalized shares that become sufficiently small.

At the beginning of every DRI iteration HAOS draws one configuration across five levels. Level one selects the number of subclusters k from a scale-adaptive domain. Its base values are clipped to the instance’s minimum feasible fleet size and extended geometrically for larger instances. The number of arms is capped at ten to allow for convergence to occur within a reasonable wallclock time. Level two selects the demand weight λ_q from six evenly spaced values between zero and one; this value controls how strongly differences in customer demand influence the decomposition. Level three selects whether customers are clustered directly (the

vertex-based paradigm) or through the routes of the current incumbent solution (the route-based paradigm). Level four selects a clustering method from the wheel associated with the chosen paradigm, with five vertex-based and three route-based methods in the deployed configuration. This conditional fourth level makes the selection hierarchical since the selection on level four directly depends on level three. Level five selects one of four routing solvers and applies it to every subcluster generated during the iteration. The first four levels therefore configure decomposition, and the fifth configures subproblem routing.

Two feasibility adjustments apply before the configuration is used; Algorithm 1 gives the draw and update in full. Route-based clustering requires an incumbent solution. Without one, HAOS uses the vertex-based paradigm and redraws the method from the vertex wheel; these applied choices receive the subsequent credit. If k exceeds the number of incumbent routes under route-based clustering, the applied k is clipped to the route count. The immediate update retains the index of the k arm originally drawn, while route tags record the configuration that was applied. Each iteration is logged with the complete operator selection and the product of the five selected probabilities, which reports the joint probability of the realized configuration under the current wheels.

HAOS maintains six roulette wheels for the five levels, because of the conditional clustering method level. Each wheel tracks one weight per option. A reward r adds to the weight of every arm selected for the credited configuration. Once per iteration, every weight is also multiplied by a common decay factor. This uniform decay is the only temporal mechanism in the weights. Both operations retain the starting weight as a lower bound:

$$w_i \leftarrow \max(w_i + r, w_0), \quad (4.1)$$

$$w_i \leftarrow \max(\gamma w_i, w_0), \quad \gamma = 0.95 \text{ by default}, \quad (4.2)$$

with every wheel starting from the same weight $w_0 = 10.0$. Before an arm is drawn, HAOS first computes its normalized weight $\hat{p}_i = w_i / \sum_l w_l$. A level-specific floor is applied to these values, followed by renormalization:

$$p_i = \frac{\max(\hat{p}_i, p_{\min})}{\sum_j \max(\hat{p}_j, p_{\min})}. \quad (4.3)$$

The floor parameter is the minimum probability any weight of any wheel can decay to. For the five levels, the floors are 0.025 for k , 0.05 for λ_q , the clustering methods, and the solver, and 0.10 for the paradigm respectively. Because Equation 4.3 renormalizes after flooring, these values are nominal pre-normalization floors rather than strict lower bounds on the final probabilities. HAOS retains three elements of the adaptive layer proposed by Ropke and Pisinger (2006) and used for general vehicle routing by Pisinger and Ropke (2007): initially equal weights, quality-dependent scores, and roulette selection proportional to the resulting weights. Its additive re-

ward, global decay, and six-wheel hierarchy are novelties introduced for DRISPI.

HAOS adapts its weights by receiving rewards from later stages of the current or following iterations. First, there is an immediate channel for results produced directly by the current DRI iteration and a deferred channel for routes that prove useful during later pool recombination. Table 4.1 gives the deployed reward values. Both channels compare a candidate with the incumbent best cost and the previous iteration’s cost, but the deferred scores are smaller because several earlier configurations can contribute routes to the same recombined solution.

Table 4.1: Default Immediate and Deferred HAOS Rewards.

Outcome	Immediate	Deferred
New incumbent best	10.0	7.0
Improvement over previous iteration only	4.0	3.0
No improvement	1.0	0.0
No solution	0.0	0.0

The immediate channel scores two candidates: the solution obtained after decomposition and subproblem routing, and the solution obtained after BG-AILS. Both rewards are credited to the same five-level configuration drawn for that iteration and therefore accumulate. Each selected arm receives the full reward. By doing so, HAOS learns from the performance of the complete configuration but does not isolate the contribution of one level or model interactions between levels.

Algorithm 1: HAOS: five-level draw and weight update for one DRI iteration

Input : wheels $W_1, W_2, W_3, W_{4v}, W_{4r}, W_5$; incumbent route count m_{inc} ;
 floors $p_{\text{min}}^{(1)}, \dots, p_{\text{min}}^{(5)}$; decay γ ; base weight w_0

Output: applied configuration $(k, \lambda_q, \text{paradigm}, \text{method}, \text{solver})$

```

1 draw  $k, \lambda_q, \text{paradigm}, \text{solver}$  from  $W_1, W_2, W_3, W_5$  // Eq. 4.3
2 if  $\text{paradigm} = \text{route}$  then
3   | draw method from  $W_{4r}$ 
4 else
5   | draw method from  $W_{4v}$ 
6 credited  $\leftarrow (k, \lambda_q, \text{paradigm}, \text{method}, \text{solver})$  // as drawn
7 if  $\text{paradigm} = \text{route}$  and  $m_{\text{inc}}$  undefined then
8   | paradigm  $\leftarrow$  vertex; redraw method from  $W_{4v}$ ; overwrite it in credited
9  $k_{\text{applied}} \leftarrow k$ 
10 if  $\text{paradigm} = \text{route}$  and  $k > m_{\text{inc}}$  then
11   |  $k_{\text{applied}} \leftarrow m_{\text{inc}}$  // credited keeps the drawn  $k$ 
12 run decomposition and routing under  $(k_{\text{applied}}, \lambda_q, \text{paradigm}, \text{method}, \text{solver})$ 
13 tag every produced route with this applied configuration
14 foreach  $\text{candidate} \in (\text{decompose-route solution}, \text{BG-AILS solution})$  do
15   |  $r \leftarrow$  reward of candidate from Table 4.1
16   | foreach arm  $i \in \text{credited}$  do
17     |  $w_i \leftarrow \max(w_i + r, w_0)$  // Eq. 4.1
18 foreach wheel  $W \in \{W_1, W_2, W_3, W_{4v}, W_{4r}, W_5\}$ , arm  $i \in W$  do
19   |  $w_i \leftarrow \max(\gamma w_i, w_0)$  // Eq. 4.2

```

Lines 10-11 are the feasibility clipping that matters for correctness: when route-based clustering draws a k larger than the incumbent route count, only k_{applied} on line 11 changes; the credited configuration fixed on line 6 keeps the originally drawn k -arm. Lines 14-17 are the reward credit itself: both the decompose-route candidate and the BG-AILS candidate (line 14) add their reward (line 15) to every arm in credited (lines 16-17), so a single iteration's two immediate rewards accumulate on the same five arms regardless of whether k had to be clipped on line 11. This is why clipping does not by itself distort the immediate update: the arm being rewarded is the one HAOS drew on line 6, not the one the pipeline used after line 11.

The deferred channel is evaluated after SC/SP has selected routes from the pool and the resulting solution has received its unrestricted improvement pass. Every stored route carries a tag containing the complete configuration under which it was generated. The deferred reward is credited once to every distinct tag among the routes selected by SC/SP, even if those tags originate from much earlier iterations. Routes created only during the unrestricted improvement pass are excluded because no HAOS selection produced them. Deferred attribution is necessary because the usefulness of one stored route may become visible only when it is combined with

routes generated later.

During the first 10 iterations, HAOS performs the same five-level draw but suppresses both reward updates and decay. All weights therefore remain equal, making every option uniform within the wheel from which it is eligible to be drawn. This warm-up period lets the first incumbent and an initial route pool form before early outcomes influence later probabilities. Learning begins with iteration 10, after which the immediate and deferred updates follow the same weight system.

4.2 Decomposition

The decomposition layer runs at the beginning of every iteration in the DRI loop, adapting the decompose-route-improve terminology of Kerscher and Minner (2025), who propose the framework for large-scale vehicle routing problems with time windows that this work extends to the CVRP at very-large scale in a fully parallelized, self-tuning setting. HAOS (Section 4.1) controls this stage through four levels, redrawn at every iteration: first the number of subclusters k ; second the clustering paradigm, either vertex-based, which partitions the customers directly, or route-based, which partitions the routes of the current incumbent solution and propagates the resulting labels to their customers; third the clustering method used within that paradigm; and fourth the demand weight λ_q of the dissimilarity metric utilized a subset of clustering approaches. Additionally, a fifth quantity that also feeds the metric, the angular offset ϕ , is drawn uniformly from $[0, 2\pi)$ at every iteration independently of HAOS. ϕ ensures that the wraparound point described below does not stay fixed across iterations.

For the decomposition, we form a dissimilarity metric that combines three terms for every pair of customers i and j with coordinates (x_i, y_i) and (x_j, y_j) and demands q_i and q_j : their Euclidean distance, an angular term, and a demand term,

$$d_{ij} = \sqrt{(x_i - x_j)^2 + (y_i - y_j)^2 + \lambda_\theta \Delta\theta_{ij}^2} \left(1 + \lambda_q \frac{|q_i - q_j|}{Q} \right). \quad (4.4)$$

$\Delta\theta_{ij}$ is the shortest angular distance between customer i 's and customer j 's polar angle around the depot, computed with the offset ϕ introduced above. λ_θ scales the angular term to the average coordinate magnitude of the instance, so that neither term dominates across instances of different spatial extent. ϕ shifts where the $\Delta\theta_{ij}$ wraparound at 2π falls. The demand factor applies the multi-feature dissimilarity metric of Kerscher and Minner (2025), who combine spatial, temporal, and demand features, here to a setting without time windows, utilizing only the demand ratio $\lambda_q |q_i - q_j|/Q$ remains alongside spatial features.

Table 4.2: Inputs Used By Each Decomposition Method.

Method	Paradigm	Input	Angular	Demand
K-Means	vertex	raw coordinates	–	–
Fuzzy C-Means	vertex	raw coordinates	–	–
K-Medoids	vertex	dissimilarity matrix	✓	✓
Agglomerative (Average)	vertex	dissimilarity matrix	✓	✓
Agglomerative (Complete)	vertex	dissimilarity matrix	✓	✓
K-Means	route	route centroids	–	–
Agglomerative (Average)	route	route centroids	–	–
Agglomerative (Complete)	route	route centroids	–	–

Both clustering paradigms trace back to the classic cluster-first, route-second scheme for capacitated routing (Gillett and Miller, 1974; Fisher and Jaikumar, 1981). Vertex-based clustering partitions the customers directly and offers five methods (Table 4.2). K-Means (MacQueen, 1967) and Fuzzy C-Means (Bezdek et al., 1984) cluster on raw two-dimensional coordinates, Fuzzy C-Means hardened to a partition by assigning each customer to its highest-membership cluster. The remaining methods K-Medoids, partitioning around medoids (Kaufman and Rousseeuw, 1990), and Agglomerative Hierarchical Clustering under average or complete linkage (Kaufman and Rousseeuw, 1990) instead cluster directly on the dissimilarity matrix of Equation 4.4. They are the only three method arms that are influenced by λ_q and ϕ .

Route-based clustering represents each route by the centroid of its customers' coordinates and clusters the centroids with K-Means or with the same average or complete linkage, applied to the plain Euclidean distance between centroids rather than to the dissimilarity matrix (Table 4.2). This follows the barycenter-clustering decomposition strategy studied by Santini et al. (2023).

The clusters are then handed over in form of a set of independent subproblems to the following routing layer (Section 4.3) to be solved. The dissimilarity matrix, rebuilt from (λ_q, ϕ) , still feeds the boundary detection used at the end of the DRI loop in the BG-AILS (Section 4.4) stage regardless of which paradigm decomposition uses. Labels then propagate from routes to customers directly, since every customer belongs to exactly one route in the incumbent solution.

4.3 Routing

Once decomposition produces the k subclusters (Section 4.2), the routing stage builds one sub-instance per subcluster, each with its own internal node IDs and its own copy of the depot. It then solves all k of them in parallel, independently of each other. The fifth and last HAOS level draws the solver once per iteration and applies it

for every subcluster that iteration, so all k sub-instances are solved by the same one of four solvers. Each sub-instance receives a time budget of $\max(\text{floor}, \text{size} \times \text{rate})$ seconds, with a default rate of 0.06 seconds per customer and a floor of 5 seconds, so small subclusters are not starved of search time. The k sub-instances are dispatched to a pool of worker processes pinned to the cores reserved for this stage; if k exceeds the worker count, the remainder queue in further waves. Section 4.7 details how this pool fits into the wider core split. A wall-clock timeout derived from a fitted wave-time model guards against a hung worker.

The four solvers are monolithic, full-instance CVRP heuristics that DRISPI treats as black boxes: AILS-II (Máximo et al., 2024)¹, an adaptive iterated local search with an elite-solution pool for large instances that extends an earlier adaptive ILS hybridized with path-relinking (Máximo and Nascimento, 2021), runs as a Java subprocess; FILO (Accorsi and Vigo, 2021)², a ruin-and-recreate local search built on the COBRA granular-neighborhood library, runs as a compiled C++ subprocess; FILO2 (Accorsi and Vigo, 2024)³, an evolution of FILO for instances up to one million customers, also a C++ subprocess; and PyVRP (Wouda et al., 2024)⁴, a Python package implementing hybrid genetic search (Vidal et al., 2012; Vidal, 2022) that DRISPI calls in-process rather than as a subprocess. The four solvers selected are not an arbitrary shortlist. Queiroga et al. (2026), who introduce the XL instance set this thesis focuses on (Chapter 3), benchmark them against several other state-of-the-art heuristics and report that AILS-II and FILO2 dominate the best-known solutions on the XL range itself, while HGS-CVRP remains the strongest performer on the smaller, medium-scale instances that a typical DRISPI subcluster is closer in size to. The selection of solver level spans exactly this cluster size spectrum.

Once the routes returned by the k individually solved subclusters are concatenated, they form one combined solution for the full instance. Because the subclusters partition the customers, no route returned at this stage spans two subclusters, and the concatenation needs no reconciliation. Each route is tagged with the subcluster of its customers, information that BG-AILS (Section 4.4) reuses once its perturbations start moving customers across subcluster boundaries. This combined, subcluster-tagged solution is the input BG-AILS perturbs and re-optimizes next.

4.4 Boundary-Guided Adaptive Iterated Local Search

The decomposition introduces structural local optima by drawing partitions that are, from the routing problem’s point of view, potentially arbitrary: a cluster boundary

¹<https://github.com/INFORMSJoC/2023.0106>

²<https://github.com/acco93/filo>

³<https://github.com/acco93/filo2>

⁴<https://pyvrp.org>

sits wherever the drawn clustering method happened to cut, not necessarily wherever the customer geometry would suggest a natural route boundary. Each subcluster can be routed to a good local optimum in isolation (Section 4.3), yet the concatenated solution can still carry structural defects that only exist across the concatenation seams: routes on either side of a boundary that would be cheaper reconnected across it, or customers left on the wrong side of an arbitrary cut. A plain, unrestricted improvement pass over the whole solution could in principle fix these defects, but under a short iteration time budget it spends most of that budget on customers that already sit comfortably inside one subcluster’s interior, where the decomposition did nothing that needs correcting. Boundary-Guided Adaptive Iterated Local Search (BG-AILS) is DRISPI’s answer to this asymmetry: an improvement stage that reuses AILS-II’s own local search but spends its comparatively short time budget disproportionately on the boundary regions the decomposition itself created.

At its core, BG-AILS is AILS-II (Máximo et al., 2024), the same monolithic solver DRISPI can also draw as a HAOS routing arm (Section 4.3), run here purely as an improvement operator rather than a cold-start solver. Two additions to stock AILS-II, both introduced for this thesis, make that role possible: a `-initialSolution` flag that seeds the search with the combined post-decompose-route solution instead of building one from scratch, and a `-initialOmega` flag that overrides the starting value of ω , AILS-II’s own diversification-control parameter governing how strongly its internal perturbation cycle disturbs the incumbent before every local-search pass (Máximo and Nascimento, 2021). Seeding AILS-II with the concatenated solution is not sufficient on its own, however: a local search that starts from that solution and only ever considers its immediate neighborhood tends to repair it straight back toward the same seam defects the decomposition introduced, because nothing has told it that the seams, rather than a subcluster’s interior, are where a profitable move is more likely to be waiting. BG-AILS therefore precedes the AILS-II call with a purpose-built perturbation step, implemented in Python, that displaces customers specifically across the boundaries between subclusters, using a single, large-neighborhood move rather than the small single-customer or single-edge moves a subsequent local search pass could trivially undo in one or two steps. This mirrors the classical iterated-local-search argument, introduced for the double-bridge move on the traveling salesman problem (Martin et al., 1991) and generalized since (Lourenço et al., 2003), that perturbation strength must be calibrated so that the following local search neither collapses straight back into the same local optimum nor is reduced to an effective random restart. No formal proof establishes this property for an arbitrary local search and perturbation pair; the argument, for the double bridge as for cross-reconnect, is structural rather than a theorem: both disconnect the current solution through a simultaneous multi-edge change that a local search built from smaller, incremental moves does not consider as a single step, and so is unlikely to reverse in one move of its own.

The perturbation step reuses the dissimilarity matrix built for the decomposition

stage (Section 4.2) rather than defining a separate notion of distance, so that “boundary” means the same thing to BG-AILS as it did to whichever clustering method drew the partition. Algorithm 2 gives the full perturbation procedure, from boundary scoring through the chain of cross-reconnects, ending at the AILS-II call itself. For every customer i with cluster label $\kappa(i)$ under the current partition, BG-AILS first scores how close i sits to a customer outside its own cluster,

$$b_i = 1 - \frac{\min_{j: \kappa(j) \neq \kappa(i)} d_{ij} - d_{\min}}{d_{\max} - d_{\min}}, \quad (4.5)$$

$$\tilde{b}_i = \frac{b_i}{|\kappa(i)|} \left(1 + \alpha \max\left(0, \frac{\sigma - |\kappa(i)|}{\sigma}\right) \right), \quad (4.6)$$

where $d_{\min}$ and $d_{\max}$ are the smallest and largest nearest-other-cluster dissimilarity across all customers, so a customer with a nearby neighbor in a different cluster scores close to one and a customer deep inside its cluster scores close to zero. Equation 4.6 divides by the size $|\kappa(i)|$ of i ’s own cluster, so boundary attention is not monopolized by whichever cluster happens to contribute the most customers near a seam, and multiplies by a boost that grows as the cluster shrinks below a small-cluster cap σ (20 customers by default) up to a factor of $1 + \alpha$ ($\alpha = 0.5$ by default), so the very small clusters route-based decomposition can produce are not perturbed less just because they contribute few customers in absolute terms. After renormalizing $\tilde{b}_i$ to $[0, 1]$ via min–max to obtain $\hat{b}_i$, a boundary threshold τ gates the result,

$$\rho_i = \frac{\max(0, \hat{b}_i - \tau)}{1 - \tau}, \quad \tau = 0.5 \text{ by default}, \quad (4.7)$$

setting ranks at or below τ to zero and rescaling the remainder linearly back onto $[0, 1]$, structurally the same floor-then-renormalize pattern HAOS applies to its wheel probabilities (Equation 4.3), just against a fixed threshold rather than a per-level minimum share. This is the sense in which BG-AILS is more efficient than an unrestricted improvement pass under the same time budget: with $\tau = 0.5$ it never spends perturbation effort on the lower half of customers by boundary rank, in particular never on a customer sitting deep inside a cluster’s interior, and instead concentrates entirely on the customers the decomposition itself flags as most likely to be sitting on a defect it created.

Boundary ranks are defined per customer, but the only perturbation BG-AILS applies acts on whole routes at once: a cross-reconnect move that cuts two routes at a random position each and exchanges their tails, so routes r_i and r_j cut after position a and b respectively become $r_i[:a] \frown r_j[b:]$ and $r_j[:b] \frown r_i[a:]$. This is the only perturbation operator BG-AILS implements. It is a large-neighborhood move by construction, since it can reassign an arbitrarily long tail of customers between two routes in a single step, which is precisely what keeps it from being trivially undone by the small single-customer and single-edge moves of a subsequent local search pass. A route’s own weight for perturbation is the maximum boundary

rank among its customers, and the first route of a cross-reconnect pair is drawn from these weights, either by roulette-wheel sampling or, under DRISPI's default greedy pair-selection setting, by taking the arg-max outright. Its partner is not drawn from the same weights again but from a per-route affinity toward each other cluster – the inverse of the mean dissimilarity between that route's customers and a given cluster's customers, row-normalized to sum to one – used first to pick which neighboring cluster to reach into from the first route's own affinity row, then to pick, among the routes majority-assigned to that cluster, the one with the highest affinity back toward the first route's own cluster. This two-step draw is what keeps a cross-reconnect pair spatially adjacent across one real cluster boundary, rather than pairing two routes that both happen to contain a boundary customer but face away from each other. BG-AILS repeats this draw-and-reconnect step for a chain of k iterations by default, one per subcluster (a $k - 1$ variant is also implemented but is not the deployed default), with each step in the chain acting on the routes as already modified by the previous one. When decomposition realizes only a single cluster, there is no boundary to guide perturbation toward, and BG-AILS skips the perturbation step entirely, falling back to a plain AILS-II improvement pass on the unperturbed combined solution.

Algorithm 2: BG-AILS: boundary-guided perturbation preceding the AILS-II call

Input : routes $\mathcal{S} = \{r_1, \dots\}$; dissimilarity matrix d_{ij} ; labels $\kappa(\cdot)$; subcluster count k ; parameters σ, α, τ

Output: perturbed solution, passed to the AILS-II call

```

1 if  $k = 1$  then
2   return  $\mathcal{S}$  unperturbed           // single-cluster fallback
3 foreach customer  $i$  do
4    $\left|$  compute  $b_i$  (Eq. 4.5) and  $\tilde{b}_i$  (Eq. 4.6)
5   min-max normalize  $\tilde{b}_i$  over all customers to obtain  $\hat{b}_i$ 
6 foreach customer  $i$  do
7    $\left|$   $\rho_i \leftarrow \text{Eq. 4.7}$            // boundary rank
8 foreach route  $r \in \mathcal{S}$  do
9    $\left|$   $\rho(r) \leftarrow \max_{i \in r} \rho_i$        // route weight
10 for  $t \leftarrow 1$  to  $k$  do
11    $r_a \leftarrow \arg \max_{r \in \mathcal{S}} \rho(r)$        // greedy by default
12    $c \leftarrow$  cluster drawn from  $r_a$ 's affinity row  $A_{r_a, \cdot}$  // affinity row
13    $r_b \leftarrow \arg \max \{A_{r, \kappa(r_a)} : r \in \mathcal{S}, \text{majority cluster}(r) = c\}$  // highest
      affinity back
14   cut  $r_a$  after random position  $p_a$ ,  $r_b$  after random position  $p_b$ 
15    $r_a, r_b \leftarrow r_a[:p_a] \cup r_b[p_b:], r_b[:p_b] \cup r_a[p_a:]$  // cross-reconnect
16   update  $\mathcal{S}$  with the reconnected  $r_a, r_b$ ; recompute  $\rho(r_a), \rho(r_b)$ 
17 call AILS-II on  $\mathcal{S}$  with -initialSolution and -initialOmega

```

Lines 12-13 are the partner draw that keeps a cross-reconnect pair spatially meaningful: the first route (line 11) is the greedy arg-max over route weights, but its partner is not drawn from those same weights again, it is drawn in two steps from r_a 's own affinity row, first to a neighboring cluster (line 12) and then to the route inside that cluster with the highest affinity back (line 13). This is why the operator pairs routes across one real boundary rather than two routes that both happen to touch a boundary customer without facing each other. The chain compounds because line 16 recomputes $\rho(r_a)$ and $\rho(r_b)$ after every reconnect before the loop re-enters lines 11-13, so each of the k steps can select a route another step in the same call already changed, rather than k independent draws against the original partition.

Table 4.3: BG-AILS Parameters and Their Defaults.

Parameter	Default	Role
Boundary thresh- old τ	0.5	Gates boundary ranks (Eq. 4.7); ranks at or below τ are zeroed.
Small-cluster cap σ	20	Clusters below this size receive a boost in Eq. 4.6.
Small-cluster boost α	0.5	Maximum boost factor $1 + \alpha$, approached as cluster size $\rightarrow 0$.
Chain-count mode	k	Number of cross-reconnect chains per call ($k - 1$ also implemented).
Pair selection	greedy	Deterministic arg-max vs. stochastic roulette-wheel route-pair draw.
Initial omega ω_0 (-initialOmega)	0.8	Overrides AILS-II's own starting diversification-control value.
Budget floor	60 s	Minimum AILS-II wall-clock limit regardless of the predicted model below.
Budget margin	1.0	Multiplier applied to the predicted decompose-route wall time.

Table 4.3 summarizes the parameters this stage introduces on top of stock AILS-II. Of these, only the initial-omega override reaches into AILS-II itself; every other AILS-II hyperparameter governing its own local search and diversification control (the parameters $d_{\min}$, $d_{\max}$, γ , and φ of Máximo and Nascimento (2021)) is left at AILS-II's own defaults. The remaining parameters – the boundary threshold, the small-cluster cap and boost, the chain-count mode, and the pair-selection rule – are new and govern only the Python-side perturbation step described above.

The key difference between BG-AILS and a plain AILS-II improvement pass is therefore not the solver itself but this focus on boundary regions, imposed both by the perturbation step and by the short wall-clock budget BG-AILS is given. That budget follows neither the fixed per-customer rate used for subcluster solving (Sec-

tion 4.3) nor a plain function of instance size, but a fitted model of the surrounding decompose-route stage’s own wall-clock time,

$$\text{budget} = \max(60\text{s}, 1.0 \times \hat{T}_{\text{DR}}), \quad (4.8)$$

$$\hat{T}_{\text{DR}} = 0.976 \times n_{\text{waves}}^{-0.180} \times \sum_{\text{waves}} \max(\text{per-cluster budgets in wave}), \quad (4.9)$$

refit on 835 post-strip pilot DRI iterations, where waves pack the current iteration’s per-subcluster solver budgets (Section 4.3) into the granted DRI worker count in partition-submit order. This choice is deliberate: DRISPI can run BG-AILS in an asynchronous mode where a dedicated worker process perturbs and improves the current iteration’s solution while the main process moves straight on to decomposing and routing the next iteration, with a single-slot handoff bounding the resulting lag to exactly one iteration. Sizing the BG-AILS budget after the very decompose-route stage it overlaps with is what keeps that overlap efficient rather than leaving either side idle; Section 4.7 covers the resulting core split and this asynchronous handoff in full. By default, however, BG-AILS instead runs synchronously within the DRI iteration, still under the same Equation 4.8 budget.

Under this comparatively short wall-clock budget, we expect BG-AILS to outperform an equally time-limited, unguided AILS-II pass over the same combined solution, precisely because the latter would spend an unknown and likely large share of that budget searching subcluster interiors the decomposition had already solved well. This comparison has, however, not yet been run; validating it on a small benchmark, holding the instance, the combined input solution, and the wall-clock budget fixed across both configurations, remains open work.

BG-AILS closes the DRI loop: its output routes are contributed to the route pool like every other stage’s output (Section 4.5), and the next iteration begins with a fresh HAOS draw over the decomposition and routing levels (Section 4.1). Periodically, rather than continuing the DRI loop for another iteration, DRISPI instead steps back and recombines the pool accumulated across many such iterations through set covering or set partitioning, the subject of the next section.

4.5 Route Pool and Set Partitioning

Periodically, DRISPI steps back from the current incumbent and recombines every route accumulated so far into one new candidate solution, using set covering or set partitioning (SC/SP) as the recombination model. Alternating a metaheuristic with an exact set-partitioning solve in this way is an established matheuristic pattern (Jourdan et al., 2009). Cavaliere et al. (2022) apply essentially this pairing to large-scale CVRP instances directly, by alternating LKH-3 local search with a restricted set-partitioning solve. Doing so only pays off if a shared memory of routes exists for it to recombine, so this section first describes that memory, the route pool, before

turning to the SP and SC models themselves and the hyperparameter that decides between them.

The route pool stores one entry per distinct customer set $C_r \subseteq C$, where C is the customer set of the instance (Chapter 3), keyed by the frozen customer set rather than by the ordered sequence of customers a route visits them in: two routes serving the same customers in a different order, or at a different cost, are treated as the same column, and the cheaper of the two overwrites the other. This column-oriented memory, populated online during the DRI loop and periodically recombined through a partitioning model, follows the adaptive-memory principle of Rochat and Taillard (1995), who introduce the route pool and its set-partitioning recombination for vehicle routing.

Two pathways feed the pool every iteration, both through the same contribute-before-adopt rule: a candidate solution is added to the pool before the pipeline decides whether to adopt it as the new incumbent, so a losing candidate still leaves its routes behind for later reuse. The DRI loop feeds the pool at least twice per iteration, once from its decompose-route stage (Section 4.3) and once from the following improvement stage (BG-AILS (Section 4.4)). The SC/SP step covered in this section feeds the pool only indirectly. Solving SC or SP selects among the customer sets already in the pool and creates no new ones, so its own output adds nothing. The post-SC/SP-improvement pass that polishes its result (Section 4.6) is what can move customers between routes and through that generate new columns. Every contributed route carries a tag recording the full HAOS selection that produced it, that is, k , the demand weight, the paradigm, the method, and the solver, with a separate flag marking routes from the post-SC/SP polish; the tag survives even once HAOS has moved on to a different selection, which is what later lets reward attribution and pool inspection trace a route back to its origin. Whenever a candidate is adopted as the new incumbent, its routes, already in the pool by the rule above, are marked elite; elite routes are exempt from eviction, so the pool never loses the one partition it is guaranteed to already have, namely the incumbent's own.

Every SC/SP solve also updates two running scores on each pool entry: A quality and a diversity score. The quality score represents a column's usefulness for the solver during the SC/SP solve. Therefore, an entry never touched by a solve carries no quality score yet. The diversity score is the mean Jaccard distance (Jaccard, 1901) between a route's customer set and every other pool entry's customer set,

$$\bar{J}(r) = \frac{1}{|R| - 1} \sum_{r' \in R \setminus \{r\}} \frac{|C_r \Delta C_{r'}|}{|C_r \cup C_{r'}|}, \quad (4.10)$$

where R is the pool at the time of the update, recomputed for every entry whenever any solve happens. New quality or diversity scores are appended to a three-value window, from which final scores are read as the mean of their respective window. This ensures that single unusually high or low values do not dominate a route's

standing.

The route pool has a maximum size (10,000 columns by default), to keep the computational complexity from blowing up. We therefore establish an eviction policy that ranks the non-elite entries on both scores and combines the ranks into one fitness value. Among the entries that have been scored at least once, ascending order by mean quality score gives rank one to the weakest route and the highest rank to the strongest; independently, ascending order by mean diversity score gives rank one to the most redundant route and the highest rank to the most unique. Writing $\rho_q(r)$ and $\rho_d(r)$ for these two ranks, fitness is $\Phi(r) = \rho_q(r) + w_d \cdot \rho_d(r)$, with the diversity weight w_d (0.5 by default) controlling how much redundancy counts against a route relative to raw quality. The entries with the lowest $\Phi(r)$ are evicted first. Because both terms grow toward the same end of their respective orderings, a route that the LP relies on heavily but that is also nearly identical to several other pool entries can rank as a stronger eviction candidate than a weak but singular one, because redundancy drives the diversity term up regardless of quality. The eviction policy only triggers once the pool exceeds its maximum size and then only removes the overflow amount. Unscored entries are evicted only if the overflow cannot be covered by scored, non-elite entries alone. Elite entries are categorically exempt from eviction.

The recombination model itself treats the pool as a column set. Let R be the set of routes currently in the pool, c_r the cost of route $r \in R$, that is, the Euclidean depot-customers-depot round trip over the travel costs of Chapter 3, and $x_r \in \{0, 1\}$ a decision variable selecting route r . Set partitioning, in the formulation Balinski and Quandt (1964) introduce for exactly this delivery-routing setting, reads

$$\min \sum_{r \in R} c_r x_r \quad (4.11)$$

$$\text{s.t.} \quad \sum_{r \in R: i \in C_r} x_r = 1, \quad \forall i \in C \quad (4.12)$$

$$x_r \in \{0, 1\}, \quad \forall r \in R. \quad (4.13)$$

Solving Equations 4.11–4.13 recombines route fragments that have never been jointly considered, for example because they may have entered the pool from different subclusters, different HAOS selections, or different iterations, into one coherent solution that still covers every customer exactly once. This is the sense in which SP looks across the boundaries any individual DRI iteration’s decomposition imposes, something no individual subcluster solve or BG-AILS perturbation can do on its own.

DRISPI builds this model once in Gurobi⁵ and solves it twice. It first relaxes Equation 4.13 to the continuous $x_r \in [0, 1]$ and solves the resulting linear program. This LP relaxation allows the solver to assign fractional weights rather than the binary

⁵<https://www.gurobi.com>

ones to each route, making it significantly easier and faster to solve. The resulting fractional weights are then utilized as the quality scores fed back into the route pool above, so every SC/SP solve doubles as a scoring pass over the pool regardless of what it ultimately selects. This follows the same spirit in which column-generation approaches read pricing information out of LP duals (Lübbecke and Desrosiers, 2005), even though DRISPI's route pool is populated by heuristic solves rather than a pricing subproblem. For the second solve, the variables are then switched to binary, warm-started from the LP solution, and re-optimized as a mixed-integer program under a wall-clock time limit and a relative optimality gap (720 seconds and 0.025% by default). A wallclock time limit is required to cut off the search with a good, if not provably optimal, solution rather than run to completion.

The equality constraint of Equation 4.12 is only a meaningful choice if the pool offers more than one real alternative for most customers. A pool still dominated by a single incumbent partition would just reproduce it again and again under SP. DRISPI therefore checks, before every SC/SP trigger, whether every customer in the instance is covered by at least a fixed minimum number of pool routes (five by default). Only when that holds does SP run. Otherwise DRISPI falls back to the less constrained set covering (SC), the inequality-constrained relaxation of the same formulation (Garfinkel and Nemhauser, 1969), whose formal problem statement and LP relaxation are given in Appendix 1. SC only requires every customer to be covered at least once rather than exactly once, a condition the pool always satisfies, because the elite routes described above are never evicted and already form a feasible partition on their own. This is also why the fallback is safe and not merely convenient: SP is never blocked by infeasibility, only by a pool that has not yet accumulated enough alternatives, and SC exists to give the SC/SP step something well defined and always solvable to run on every scheduled trigger, warmup aside, no matter how much the pool has matured. The price of that permissiveness is that SC's selected routes can overlap on a customer, meaning a customer can be visited by more than a single route and therefore more than exactly once. Since this would violate our problem statement formulated in Section 3, a savings-based deduplication heuristic then removes each duplicated customer from every selected route except the one where removing it would save the least cost, restoring a feasible partition.

A solved partitioning problem will pair routes into a new candidate solution which haven't been evaluated together before. And while search heuristics have been employed in identifying the routes used, the resulting new set of routes has not been polished by evaluating for example inter-route moves amongst each other. To do just that the SC/SP step is combined with a standard-AILS pass (Section 4.6) to improve the partitioning result even further if possible.

4.6 Improvement

Once SC/SP (Section 4.5) recombines the route pool into one partition, DRISPI runs one further pass over it using AILS-II as solver, seeded only with that partition as an initial solution. Unlike in BG-AILS (Section 4.4), this step focuses less on perturbation but rather refinement by local search. This standard improvement step also allows the solver to touch all routes instead of biasing moves towards certain neighborhoods. The pass runs under its own wall-clock budget (360 seconds by default) on the same core reservation as the SC/SP solve that precedes it; Section 4.7 covers how exactly a parallel architecture of DRISPI looks.

The refined routes feed back into the route pool exactly like every other stage's output (see Section 4.5): a route whose customer set matches one SC/SP already selected keeps that route's existing tag, while a route AILS-II creates by moving customers between routes is added as new, tagged as an improvement route rather than credited to a HAOS selection of its own.

This is also where the deferred half of HAOS's reward system (Section 4.1) closes. The pass's outcome, a new best, an improvement over the previous iteration without beating the incumbent, or neither, is scored through the deferred branch of HAOS's reward function and credited not to the current iteration's own draw but to every distinct HAOS tag the routes SC/SP selected already carried, that is, to whichever earlier decompose-route or BG-AILS draws produced those specific routes, regardless of how many iterations ago that was. Improvement-route tags are excluded from this credit, since AILS-II created them without any HAOS draw of their own to reward.

4.7 Pipeline Orchestration

The preceding sections describe HAOS, decomposition, routing, BG-AILS, the route pool and SC/SP, and the post-SC/SP improvement pass as if the DRI loop executed them one after another, the sequential picture Figure 4.1 shows. DRISPI instead runs three of these stages as separate worker groups sharing one CPU allocation, so that a DRI iteration and the BG-AILS or SC/SP work left over from an earlier one can occupy the same wall-clock interval instead of queuing behind each other. This section describes that parallelized pipeline.

DRISPI splits its CPU allocation into three disjoint worker groups: `dri` cores for the subcluster routing pool of Section 4.3, `bg` cores for the BG-AILS worker of Section 4.4, and `sp` cores for the SC/SP and post-SC/SP improvement worker of Sections 4.5 and 4.6. The deployed split for the 100-instance campaign of Chapter 5 is 6+1+1 inside each 8-core Docker container (Section 5.1.1), the `cores` block of `configs/final_benchmark.yaml`; the `split` is a configuration value, not a fixed architectural constant, and a deployment with more cores per container widens

dri rather than the other two groups, since subcluster routing is the stage with an unbounded number of independent jobs per iteration.

BG-AILS and SC/SP each support a synchronous and an asynchronous mode, and the two modes differ in the same way for both stages: synchronously, the stage runs inline inside the current DRI iteration and the loop waits for it; asynchronously, a long-lived worker process owns the stage’s reserved cores across iterations, and the main loop dispatches a job to it without waiting for the result. DRISPI’s own configuration default is synchronous for both stages, matching the sequential description of Section 4.4 and Section 4.5; the asynchronous mode is implemented for both and is what `configs/final_benchmark.yaml` enables for the campaign behind Chapter 5, since only the asynchronous mode lets the bg and sp cores do anything while the dri cores are busy on the next iteration.

The BG-AILS worker uses a single-slot handoff: at most one BG-AILS job is ever in flight. The main loop dispatches the current iteration’s combined decompose-route solution to the worker and moves straight on to decomposing and routing the next iteration on the dri cores; at the start of the iteration after that, before drawing a new HAOS configuration, the loop blocks until the worker returns, drains the result, and only then dispatches the next job. This bounds the resulting lag to exactly one iteration by construction, never more, because the loop will not dispatch a second job while the first is outstanding. Equation 4.8 sizes the BG-AILS wall-clock budget against a fitted model of the very decompose-route stage it overlaps with, rather than against instance size directly, precisely because that overlap is the point of running BG-AILS asynchronously: a budget much shorter than the next decompose-route stage would leave the bg core idle before the handoff, and a budget much longer would leave the dri cores waiting on a BG-AILS job that already had the next iteration’s input ready.

The SC/SP worker instead uses a skip overlap policy rather than a blocking handoff. When a scheduled SC/SP trigger comes due, either every `sp_interval` iterations or, as configured for the campaign, every 20 minutes of wall-clock time, DRISPI dispatches a snapshot of the route pool and the current incumbent to the SP worker only if that worker is not already busy with a previous job; if it is, the trigger is skipped outright rather than queued, and the pipeline tries again at the next scheduled trigger. This is why the SC/SP branch never stalls the dri or bg workers: a slow SP solve, the 720-second time limit of Section 4.5 is a ceiling, not a typical duration, can only cause a later trigger to be skipped, never a delay elsewhere in the loop. The SP worker owns both the SC/SP solve and the post-SC/SP improvement pass of Section 4.6 for a given job, running them one after another on the same sp-core reservation before the result becomes available for the main loop to drain.

Figure 4.2 shows this overlap across two consecutive iterations, one lane per worker group: the dri lane occupied by decomposition and routing in every iteration, the bg lane running one iteration behind it under the single-slot handoff, and the sp lane

occupied only when a trigger fires, spanning several dri iterations without blocking them.

[TODO: FIGURE: swimlane diagram over wall-clock time, one horizontal lane each for the dri, bg, and sp worker groups, spanning two consecutive DRI iterations. dri lane: back-to-back decompose+route blocks, one per iteration. bg lane: one perturb+improve block per iteration, offset to start when the dri block of the same iteration ends and to finish before the dri block of the iteration after next begins (single-slot handoff, one-iteration lag). sp lane: mostly idle, with one long solve+improve block spanning multiple dri iterations, triggered every 20 minutes of wall-clock time (final_benchmark.yaml), with an arrow marking a would-be trigger that is skipped because the previous sp block is still running. Requires the actual per-stage wall-clock durations from a representative run; not fabricated here.]

Figure 4.2: Core allocation and cross-iteration overlap in DRISPI's parallelized pipeline.

Algorithm 3 gives the resulting control loop from the main process's perspective, in the asynchronous form both stages run under for the campaign of Chapter 5; the synchronous default replaces every dispatch of a BG-AILS or SC/SP job with an inline call that blocks the loop until that job returns, collapsing the loop back to Figure 4.1's sequential picture.

Algorithm 3: DRISPI: parallelized pipeline control loop (asynchronous BG-AILS and SC/SP)

Input : instance; HAOS wheels (Algorithm 1); dri/BG/SP workers; route pool $R \leftarrow \emptyset$; incumbent $S^* \leftarrow \text{None}$; limits Γ, Π

Output: incumbent S^* at termination

```

1  $i \leftarrow 0$ 
2 while elapsed time  $< \Gamma$  and iterations since improvement  $< \Pi$  do
3   if  $i > 0$  and the BG worker is busy then
4     join (blocking): drain  $S_{\text{bg}}^{(i-1)}$ 
5     contribute its routes to  $R$ ; adopt as  $S^*$  if improving; credit immediate
      reward
6   if the SP worker has a completed job then
7     drain (non-blocking): contribute its routes to  $R$ ; adopt as  $S^*$  if
      improving; credit deferred reward per tag; free the SP worker
8    $(k, \lambda_q, \text{paradigm}, \text{method}, \text{solver}) \leftarrow \text{HAOS-Draw}()$  // Algorithm 1
9   partition  $\leftarrow$  decompose  $k$  subclusters under  $(\lambda_q, \text{paradigm}, \text{method})$ 
10  dispatch one routing job per subcluster to the dri pool
11  join: collect all  $k$  subcluster solutions
12   $S_{\text{dr}}^{(i)} \leftarrow$  concatenate them
13  contribute  $S_{\text{dr}}^{(i)}$  routes to  $R$ ; adopt as  $S^*$  if improving; credit immediate
      reward
14  dispatch  $(S_{\text{dr}}^{(i)}, \text{partition})$  to the BG worker // single-slot
      handoff
15  if SC/SP is scheduled at iteration  $i$  then
16    if the SP worker is busy then
17      skip this trigger // overlap: skip, not queue
18    else
19      dispatch a snapshot of  $(R, S^*)$  to the SP worker
20      if coverage  $\geq$  minimum in the snapshot then solve SP
21      else solve SC; deduplicate
22      run the post-SC/SP improvement pass on the result
23   $i \leftarrow i + 1$ 
24 return  $S^*$ 

```

Lines 15-22 are the scheduled SC/SP branch: the coverage check and the resulting SP-or-SC choice (lines 20-21) and the post-SC/SP improvement pass (line 22) are the same procedure Section 4.5 and Section 4.6 describe, dispatched as one job rather than run inline; line 17 is the skip that keeps a busy SP worker from ever blocking the loop above it. Lines 3-5 are the asynchronous BG-AILS handoff on the reading side: the join only happens once the worker is already busy from a previous dispatch (line 3), so the first iteration never blocks, and every iteration

after that blocks on exactly one prior job (line 4) before line 14 hands the worker a new one, which is what keeps the BG lag fixed at one iteration regardless of how the rest of the loop's timing drifts.

Lines 5, 7, and 13 are the three points where a candidate's routes enter R before the pipeline decides whether to adopt it as S^* , one per worker group, BG, SP, and DRI respectively, the contribute-before-adopt rule of Section 4.5 applied uniformly across all three: a candidate that does not become the new incumbent still leaves its routes in the pool for a later SC/SP call to draw on. The post-SC/SP improvement pass on lines 19-22 produces a fourth candidate, but because that job runs asynchronously, its contribution and adopt check happen only when line 7 drains it in a later iteration, not inline within lines 15-22 themselves.

5 Computational Study

xyz?

5.1 Experimental Design

xyz?

5.1.1 Framework Implementation

All experiments in this chapter run on a single, shared department server, Ubuntu 22.04 LTS, two Intel Xeon E5-2697A v4 CPUs at 2.60 GHz (16 cores each, 32 cores and 64 threads total) and 125 GB of RAM, with no GPU. DRISPI runs inside Docker containers (Docker 29.6.1) pinned to a disjoint CPU range with `-cpuset-cpus`, because other jobs share the same server, and every worker sets `OMP_NUM_THREADS`, `OPENBLAS_NUM_THREADS`, and `MKL_NUM_THREADS` to one; the pinned CPU count, not a library thread-pool default, therefore sets the framework's own parallelism budget.

The 100-instance, three-seed campaign behind Section 5.2.1 (300 runs, 7,200 seconds each) pins 32 of the server's 64 logical CPUs as four exclusive 8-core Docker containers, each split six DRI workers, one BG-AILS worker, and one SP worker, the `cores` block of `configs/final_benchmark.yaml`. Runs are packed into waves ordered by instance size so that the three seeds of the largest instance, XL-n10001-k1570, never share a wave; the full campaign takes 75 waves and about 150 hours of wall-clock time.

Section 4.3 introduces the four subsolvers and Section 4.5 the Gurobi SC/SP step; this section fixes the concrete versions behind them. DRISPI targets Python 3.11, the version `scikit-learn-extra`'s `k-medoids` implementation requires, with dependencies locked via `uv`: `PyVRP` 0.13.3, `scikit-learn` 1.8.0 (K-Means and agglomerative clustering), `scikit-learn-extra` 0.3.0 (`k-medoids`), `scikit-fuzzy` 0.5.0 (Fuzzy C-Means), `gurobipy` 13.0.1, and `vrplib` 2.1.0 for instance parsing. AILS-II, FILO, and FILO2 are vendored as git submodules pinned to a fixed commit each and compiled at image build time, rather than resolved from a floating branch, so a rebuild reproduces the exact binaries behind the results in Section 5.2.1. The whole framework runs inside a Docker image built from this locked environment, so the software stack behind every reported run is fixed independently of the host it executes on.

5.1.2 Dataset

Chapter 3 introduces the XL instance set, the 100 CVRP instances with which Queiroga et al. (2026) extend the 100-to-1,000-customer X set of Uchoa et al. (2017) up to 10,000 customers. Queiroga et al. (2026) generate every instance from the X set's own scheme: two-dimensional Euclidean coordinates on a $[0, 1,000] \times [0, 1,000]$ grid, with the depot Random, Central at (500, 500), or Eccentric at (0, 0); customers Random, Clustered, or Random-Clustered around a small number of seed points; demand drawn from one of seven distributions, from unitary demand (U) through four ranges of increasing width (1–10, 5–10, 1–100, 50–100) to a quadrant-dependent range (Q) and a mix of many small and few large demands (SL); and a target average route length r drawn from one of seven bands, from ultra-short ($r \in [3, 5]$) to ultra-long ($r \in [50, 200]$). Vehicle capacity Q and the vehicle count are chosen jointly so that a solution using the minimum feasible number of routes attains the target r . The instance name encodes both n and this minimum route count, for example XL-n4535-k1134 for the 4,535-customer instance with a 1,134-route lower bound; this k is fixed by the benchmark set and unrelated to the decomposition subcluster count of the same name (Section 4.1), and solutions using more than k routes remain feasible. Appendix 2 lists the depot position, customer distribution, demand distribution, Q , and r of every instance in the abbreviations just defined, alongside DRISPI's result.

Queiroga et al. (2026) establish an initial Best Known Solution (BKS) for each instance from sixty runs of eight monolithic CVRP solvers: AILS-II, FILO, FILO2, KGLSXXL, HGS-CVRP, SISRs, LKH-3, and OR-Tools. AILS-II alone supplies 93 of the 100 initial values, FILO2 six, and FILO one. Section 5.2.1 benchmarks DRISPI against the same eight solvers.

These initial values were a starting point rather than a permanent baseline. Queiroga et al. (2026) launch them alongside the CVRPLib BKS Challenge, a 30-day, community-run competition, opened January 12, 2026, in which any team can submit an improved feasible solution for an XL instance, verified automatically by the CVRPLib platform. By the current record⁶, the challenge has improved 99 of the 100 initial values; XL-n1094-k157 is the only instance still at its initial value. Every gap reported in this chapter therefore uses the current BKS record as the baseline, not the initial values Queiroga et al. (2026) report.

5.1.3 Repetitions

DRISPI runs every one of the 100 XL instances three times, $S = 3$, at the fixed seeds 10, 20, and 30, disclosed here for reproducibility, each run stopped after 7,200 seconds (120 minutes) of wall-clock time rather than at convergence. Each

⁶https://galgos.inf.puc-rio.br/cvrplib/index.php/en/bks_challenge/score/instances, accessed September 7, 2026.

run is therefore a terminating simulation in the sense of Law and Kelton (2000): a fixed initial condition, the first DRI iteration’s decomposition and routing pass (Section 4.2), and a fixed stopping rule rather than a steady state. The replication method of Law and Kelton (2000, ch. 9) therefore applies; neither warm-up deletion nor batch means do.

The interval sized below is conditional on the fixed 100-instance Queiroga XL set (n from 1,048 to 10,001 customers, Section 5.1.2): the instances are a design factor (Queiroga et al., 2026), not a sample from a superpopulation of CVRP instances, so seed is the only randomness the design controls. With $\bar{G}$ the grand mean gap and $\bar{\sigma}^2$ the average per-instance seed variance, $\text{Var}(\bar{G}) = \bar{\sigma}^2/(I \cdot S)$, $I = 100$; an unconditional target would add a between-instance term that swamps the seed term and makes S nearly irrelevant, so the conditional target is stated explicitly. The interval does not extend to the published baselines Section 5.2.1 compares against, each a single reported value per instance with no variance estimate of its own.

Independence across instances is asserted mechanistically rather than inferred from the pilot data: every run executes as a separate process with its own seeded random-number stream, and DRISPI shares no pool, cache, or global counter across instances (Chapter 4), so the $I \times S$ observations behind $\bar{G}$ are independent by construction.

The pilot campaign (three instances, four seeds each, five wall-clock marks per run) measured a root-mean-square per-instance seed standard deviation of 0.118 percentage points at the 7,200 second mark. The target margin was fixed in advance at ± 0.03 percentage points, chosen to resolve the 0.02–0.06 pp effect sizes observed in earlier component A/B tests. Table 5.1 forecasts the standard error and normal-approximation 95% half-width of $\bar{G}$ at this pilot sigma, S from one to four, $I = 100$. Every value of S clears the target by a comfortable margin, so precision alone does not select $S = 3$.

Table 5.1: Forecast standard error and 95% half-width of $\bar{G}$ by seed count S .

S	SE (pp)	Half-width (pp)
1	0.0118	0.023
2	0.0083	0.016
3	0.0068	0.013
4	0.0059	0.012

Three considerations beyond precision fix $S = 3$. First, degrees of freedom: pooled within-instance degrees of freedom, $I(S - 1)$, are zero at $S = 1$ and admit no interval at all; at $S = 3$ they are 200, reduced by Satterthwaite to approximately 138 given the pilot’s heterogeneous variances (3.7-fold in standard deviation, 13.6-fold in variance), giving $t(0.975, 138) \approx 1.98$, close to the normal approximation above. Second, identifiability: at $S = 2$ a discrepancy between two runs is measurable but not attributable; at $S = 3$ a single anomalous replication can be isolated against the

other two, a capacity the campaign exercised ([**TODO: SECTION: home for realized per-instance seed variance and per-seed breakdown – no existing section covers this yet**]). Third, best-of- S requires $S > 1$ to be a nontrivial minimum, and the convention on this instance set, following Queiroga et al. (2026), reports such a column.

The design already commits 300 runs, 7,200 seconds each, to four concurrent eight-core slices on the department server (Section 5.1.1): 150 hours of wall-clock time, 4,800 core-hours. A fourth seed would add 100 more runs, about 1,600 core-hours, for a half-width gain from 0.013 to 0.012 pp, against a systematic batch effect of approximately 0.039 pp that two pilot campaigns differing only in execution environment showed, roughly three times the forecast $S = 3$ half-width and identified in advance as the binding constraint on validity, not the replication count. Johnson (2002) and Barr et al. (1995) both argue for sizing a campaign against its dominant nuisance source rather than against statistical power alone; a fourth seed buys the latter on a term already dominated by the former.

Neyman allocation, assigning more replications to higher-variance instances, was evaluated against equal allocation at the same total budget. Despite the 13.6-fold variance spread across the pilot instances, it recovers only an 8.8% reduction in the standard error of $\bar{G}$, so equal allocation, three seeds on every instance, was retained.

Two caveats qualify this design. The seed is a replication label, not a determinism guarantee: DRISPI's `ProcessPoolExecutor`-based worker pool schedules subcluster work non-deterministically, so two runs at the same seed are exchangeable replications rather than bitwise-reproducible runs. The best-of- S statistic is an extreme-value statistic whose expectation depends on S , not comparable across publications using a different replication count; the interval above applies to the mean-over-seeds statistic $\bar{G}$ only, a distinction stated here so Section 5.2.1 can rely on it rather than restate it.

Realized seed variance and the per-seed breakdown appear in [**TODO: SECTION: home for realized per-instance seed variance and per-seed breakdown – no existing section covers this yet**]; the paired Wilcoxon signed-rank tests with Holm correction on per-instance differences, following Demsar (2006), are specified in Section 5.2.1.

5.1.4 Hyperparameters

Show the hyperparameters used for the benchmark run. Explain the impact of the different hyperparameters on the performance.

5.2 Results

Short introduction.

5.2.1 Performance Comparison against State-of-the-Art Monolithic Solvers

Table 5.2 compares DRISPI against the eight monolithic solvers Queiroga et al. (2026) benchmark on the 100 XL instances: AILS-II, FILO, FILO2, KGLSXXL, HGS-CVRP, SISRs, LKH-3, and OR-Tools. For every solver we recompute the percentage gap to the current official BKS, the record listed by the CVRPLib BKS Challenge (Section 5.1.2), for both the mean-of- N and best-of- N cost the original paper reports ($N = 60$ for the published solvers, $N = 3$ for DRISPI), then average that per-instance gap over all 100 instances and, separately, over the fifty instances with fewer than 3,400 customers and the fifty with more, the split Queiroga et al. (2026) use for their own results table. Every gap in Table 5.2 uses the current BKS as the denominator, including for the published solvers. Queiroga et al. (2026) report their own average gaps against the BKS at the time of publication, effectively AILS-II’s own result for most instances, so those published averages are not comparable to the ones reported here. Appendix 2 lists the per-instance detail behind DRISPI’s two columns: instance characteristics, best cost, and gap, alongside both BKS references (Table 6.1).

Table 5.2 sets DRISPI’s own runs against the eight published solvers under two different resource budgets, not an equalized one. DRISPI runs 7,200 seconds (120 minutes) of wall-clock time per instance-seed (Section 5.1.3), on eight cores pinned inside a Docker container on a department server shared concurrently with unrelated jobs (Section 5.1.1). Queiroga et al. (2026) report AILS-II, FILO, FILO2, KGLSXXL, HGS-CVRP, SISRs, LKH-3, and OR-Tools under **[TODO: NUMBER: Queiroga et al. per-solver time budget and hardware]**. Every gap in Table 5.2 is therefore a wall-clock-to-wall-clock comparison rather than a CPU-time-to-CPU-time one: none of the reported gaps normalize for core count, for the eight published solvers’ own single-threaded execution against DRISPI’s eight-core allocation, or for the department server’s shared load against whatever dedicated or exclusive hardware the published benchmark used. This asymmetry runs against DRISPI rather than for it: DRISPI’s wall-clock budget is the only one of the nine compared here that is shared with other work on the same physical machine, and it is the only one that spends part of every run on decomposition, cross-boundary perturbation, and periodic exact set-partitioning solves rather than on a single, un-interrupted local search.

Table 5.2: Mean percentage gap to the current BKS, DRISPI against eight published solvers, sorted by ascending mean-of- N gap over all instances.

Solver	Mean-of- N Gap (%)			Best-of- N Gap (%)		
	All	$n < 3,400$	$n > 3,400$	All	$n < 3,400$	$n > 3,400$
AILS-II	+0.16	+0.14	+0.18	+0.09	+0.06	+0.12
FILO2	+0.29	+0.29	+0.30	+0.20	+0.17	+0.23
DRISPI	+0.31	+0.24	+0.37	+0.28	+0.21	+0.34
FILO	+0.33	+0.31	+0.36	+0.24	+0.19	+0.28
SISRs	+0.58	+0.49	+0.68	+0.42	+0.30	+0.55
KGLSXXL	+1.09	+0.87	+1.31	+0.88	+0.64	+1.12
HGS-CVRP	+1.45	+0.83	+2.06	+1.20	+0.60	+1.80
LKH-3 ($n = 90$)	+6.75	+5.01	+8.92	+5.56	+4.03	+7.48
OR-Tools	+6.90	+6.77	+7.03	+5.63	+5.43	+5.83

Mean-of- N is the primary comparison in Table 5.2, because DRISPI’s mean-of-3 and each published solver’s mean-of-60 both estimate the same population quantity, the expected cost of a single run, and are just averages over different sample sizes. Best-of- N is not: it is the minimum of three draws for DRISPI against the minimum of sixty for every published solver, a sampling budget 20 times larger, which mechanically favors the columns with more draws. Sampling one of DRISPI’s own three seed costs at a time gives an expected best-of-1 gap equal to the mean-of-3 gap, 0.309%; the average of the smaller cost in each of the three retained seed pairs gives a best-of-2 gap of 0.286%; the actual three-seed minimum gives the best-of-3 gap of 0.277%, already reported in Table 5.2. The marginal gain is 0.022 pp from the first extra sample and 0.009 pp from the second. AILS-II, FILO2, FILO, SISRs, KGLSXXL, and HGS-CVRP gain 0.07, 0.09, 0.10, 0.16, 0.21, and 0.25 pp respectively between their own mean-of-60 and best-of-60 columns, larger reductions from many more repetitions. DRISPI’s mean per-instance seed standard deviation is 0.035 pp, so additional seeds narrow its best column by less than they narrow a higher-variance baseline’s. This produces the rank inversion in Table 5.2: DRISPI ranks third by mean-of- N and fourth by best-of- N , without a change in the underlying solution quality; the difference is which column rewards low run-to-run variance and which rewards a wide sampling budget. Best-of- N is reported to match the published solvers’ own headline metric, not as the primary comparison.

The means in Table 5.2 do not establish whether an observed difference is real rather than an artifact of the 100 sampled instances. We tested DRISPI’s mean-of-3 column against the corresponding mean-of-60 column of five of the eight published solvers, AILS-II, FILO2, FILO, SISRs, and HGS-CVRP, with the paired Wilcoxon signed-rank test (Demsar, 2006), treating the 100 instances as matched pairs, and

controlled the family-wise error rate across these five tests with the Holm step-down correction (Demsar, 2006). KGLSXXL, LKH-3, and OR-Tools are omitted: their gaps in Table 5.2 are large enough that the paired comparison would not be informative. Best-of- N compares DRISPI's three draws against sixty for every published solver, a sampling budget twenty times larger, and is not the equal-sample-size comparison the paired test requires; it is reported below descriptively, alongside each mean-of- N result, rather than tested.

DRISPI's mean-of-3 is significantly worse than AILS-II's mean-of-60 (+0.150 pp, DRISPI ahead on seven of 100 instances, Holm-corrected **[TODO: NUMBER: Holm-corrected p, family of 5]**; best-of- N , descriptive only: +0.188 pp, DRISPI ahead on one of 100) and significantly better than SISRs (−0.274 pp, Holm-corrected **[TODO: NUMBER: Holm-corrected p, family of 5]**; best-of- N , descriptive only: −0.148 pp). It is also significantly better than HGS-CVRP, by a wider margin (−1.139 pp, DRISPI ahead on all 100 instances, Holm-corrected **[TODO: NUMBER: Holm-corrected p, family of 5]**; best-of- N , descriptive only: −0.924 pp, DRISPI ahead on 96 of 100). These three verdicts do not depend on the exact corrected value: each was already significant at $p < 10^{-9}$ under the wider, ten-test family this paragraph previously reported, and the Holm correction on a five-test family, a strict subset of that ten, cannot raise a retained test's corrected p above what it already was under the larger family. Against FILO, the mean-of-3 versus mean-of-60 comparison is −0.025 pp, Holm-corrected **[TODO: NUMBER: Holm-corrected p, family of 5]** (best-of- N , descriptive only: +0.040 pp); against FILO2, the primary comparison, it is +0.015 pp, Holm-corrected **[TODO: NUMBER: Holm-corrected p, family of 5]** (best-of- N , descriptive only: +0.074 pp). Both were non-significant under the ten-test family; whether either crosses into significance under the smaller five-test family depends on the recomputed value and is not asserted here.

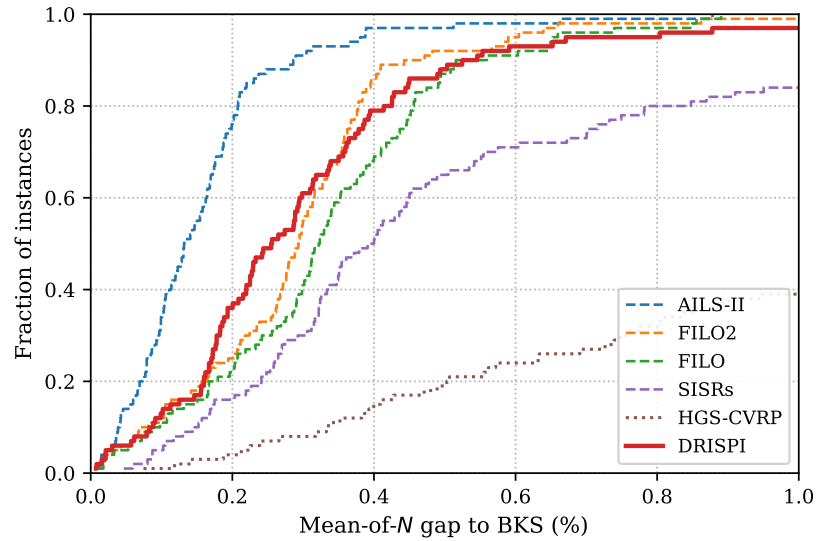


Figure 5.1: Empirical CDF of the mean-of- N gap to the current BKS. KGLSXXL, LKH-3, and OR-Tools are omitted (Section 5.2.1): their gaps would compress the visible range for the other six.

Figure 5.1 shows why the Table 5.2 means understate DRISPI's typical result and its tail at once: DRISPI's mean-of-3 gap has a mean of 0.309% against a median of 0.255% (interquartile range 0.172%–0.380%, ninetieth percentile 0.527%, maximum 1.683%), a right-skewed distribution in which most instances sit below the mean and a short tail sits well above it. DRISPI's curve tracks above FILO2's up to a gap of about 0.3% (61% of instances at or below that gap, against 55% for FILO2) and falls below it from 0.4% (79% against 86%), the crossover the non-significant paired mean-of- N test against FILO2 allows: DRISPI wins on the tighter half of the distribution and loses on the wider half. Against FILO, DRISPI's curve stays at or above FILO's over the range shown, consistent with the near-even 53-of-100 split in the paired test. HGS-CVRP's curve sits below all five other curves across the entire range shown: only 39% of its instances reach a gap of one percent or less, and it does not visibly approach the other curves anywhere in the figure, consistent with the wide, significant margin the paired test above reports against it.

Table 5.3 lists the instances behind that tail, the largest mean-of-3 gaps in Table 6.1, in descending order down to about +0.6 percent.

Table 5.3: Instances with the largest mean-of-3 gap to the current BKS (codes per Section 5.1.2).

Instance	n	Depot	Customers	Demand	r	Mean-of-3 Gap (%)
XL-n9571-k55	9,571	R	R	Q	174.7	+1.68
XL-n6034-k61	6,034	R	C (5)	5-10	98.9	+1.18
XL-n2634-k17	2,634	R	C (4)	1-10	162.6	+1.17
XL-n5174-k55	5,174	C	C (6)	1-10	94.1	+0.88
XL-n8207-k108	8,207	C	C (2)	1-10	75.9	+0.80
XL-n5902-k122	5,902	E	RC (3)	Q	48.8	+0.67
XL-n8028-k294	8,028	C	R	Q	27.3	+0.65

Five of these seven instances fall in the dataset’s ultra-long route band, $r \geq 50$ (Section 5.1.2); Section 5.2.2 takes up this concentration directly.

Two individual results do not appear in any aggregate. On XL-n4535-k1134, DRISPI’s best-of-3 cost of 1,203,553 is below the initial BKS of 1,203,566 Queiroga et al. (2026) reported at publication. On XL-n1094-k157, DRISPI’s best-of-3 cost matches both the initial and the current BKS exactly, at 112,431. Across all 100 instances, DRISPI’s best-of-3 cost matches or beats AILS-II’s mean-of-60 on 12 instances, listed in Table 5.4.

Table 5.4: Instances where DRISPI’s best-of-3 cost matches or beats a published reference.

Instance	r	DRISPI Best-of-3	Reference Cost	Reference
XL-n1094-k157	7.8	112,431	112,431.0	Initial & current BKS
XL-n1281-k29	45.5	31,161	31,201.3	AILS-II mean-of-60
XL-n1328-k19	72.6	38,261	38,317.4	AILS-II mean-of-60
XL-n1374-k278	4.9	233,247	233,248.0	AILS-II mean-of-60
XL-n1654-k11	155.4	36,420	36,429.1	AILS-II mean-of-60
XL-n1794-k163	11.2	141,739	141,777.6	AILS-II mean-of-60
XL-n1981-k13	153.5	32,597	32,639.1	AILS-II mean-of-60
XL-n2307-k34	69.6	47,980	48,020.2	AILS-II mean-of-60
XL-n2727-k546	5.3	431,165	431,173.0	AILS-II mean-of-60
XL-n3804-k29	134.0	52,930	52,935.8	AILS-II mean-of-60
XL-n3888-k1010	4.2	1,881,512	1,882,369.0	AILS-II mean-of-60
XL-n4535-k1134	4.3	1,203,553	1,203,566.0	Initial BKS

5.2.2 Performance by Instance Characteristics

The XL instance set varies two candidate notions of scale at once: the customer count n and the target average route length r (Section 5.1.2). The two are not interchangeable stand-ins for the same underlying quantity. r is not derived from n ; it is an independent design factor of the benchmark's own generation scheme, which fixes vehicle capacity Q and fleet size jointly so that a minimum-route solution attains the target r for whatever n the instance happens to have, so the 100-instance design deliberately crosses a wide range of r against a wide range of n rather than letting one determine the other. For DRISPI specifically, the two also act on the pipeline differently: n sets how many customers decomposition has to distribute across k subclusters, while r sets how many stops a single vehicle round trip covers on average and, by construction, compresses the number of routes relative to customers once it grows large. A regression that treats n as the only scale variable therefore risks crediting to instance size a pattern that a small number of long-route instances are actually driving.

A linear regression of the mean-of-3 gap on n over all 100 instances gives a slope of **[TODO: NUMBER: gap-on- n slope, all 100 instances]** (test statistic **[TODO: NUMBER: gap-on- n test statistic, all 100 instances]**, $p =$ **[TODO: NUMBER: gap-on- n p-value, all 100 instances]**, 95% confidence interval **[TODO: NUMBER: gap-on- n 95% CI, all 100 instances]**). We define the long-route instances as those in the dataset's own ultra-long route band, $r \geq 50$ (Section 5.1.2), a threshold fixed by the benchmark's band definition rather than chosen after inspecting the gaps, and it separates 14 long-route instances from the remaining 86. Restricting the same regression to those 86 instances changes the slope to **[TODO: NUMBER: gap-on- n slope, $r < 50$ only]** (test statistic **[TODO: NUMBER: gap-on- n test statistic, $r < 50$ only]**, $p =$ **[TODO: NUMBER: gap-on- n p-value, $r < 50$ only]**, 95% confidence interval **[TODO: NUMBER: gap-on- n 95% CI, $r < 50$ only]**). **[TODO: NUMBER: state here, once both p-values above are filled in, whether the n -slope loses significance on the $r < 50$ subset]**. If it does, the scale-gap relationship measured over all 100 instances is carried substantially by the 14 long-route instances rather than by instance size on its own.

This pattern is consistent with what decomposition does to a long route rather than with a direct effect of n . Every clustering method in Table 4.2 draws its partition from customer coordinates or route centroids and the dissimilarity metric of Equation 4.4, without reference to which vehicle will eventually need which customers; a long route spans a customer set spread further apart on average than a short route serving a comparable number of stops, and that same spatial spread makes it more likely that some clustering cut line passes through the route rather than around it. Once a long route straddles a cluster boundary this way, the routing stage (Section 4.3) still solves each side of the cut to a good local optimum in isolation, but the seam between them is exactly the kind of defect BG-AILS exists to repair (Sec-

tion 4.4). BG-AILS repairs every seam in an instance under the same fixed chain length of k cross-reconnects and the same wall-clock budget of Equation 4.8 regardless of how many of those seams cut through a long route rather than a short one, so an instance whose long routes cross more boundaries has more repair work competing for a budget that does not grow to match it.

Figure 5.2 sets this account against the remaining instance characteristics descriptively: depot position, customer distribution, demand distribution, and n , each summarized by the per-category mean-of-3 gap (panels a-c) or plotted directly against the per-instance gap (panel d), rather than argued from.

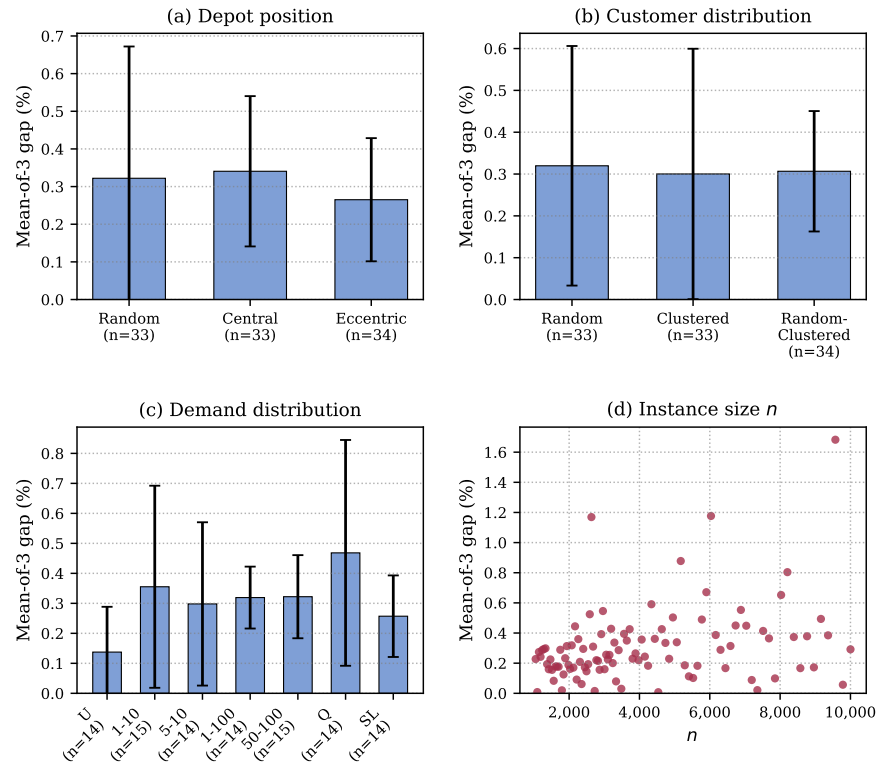


Figure 5.2: Mean-of-3 gap to the current BKS by instance characteristic, across all 100 XL instances (error bars are one sample standard deviation).

5.2.3 HAOS Weights

Investigate how HAOS weights are converging and if there are obvious tendencies. Interpret the results.

Are some HAOS levels converging always to the same result? Are some HAOS levels always converging in a similar way subject to the instance characteristics? Whats the variance in HAOS weight convergence? Do they differ a lot from seed to seed and are sensitive to randomness?

Are the results interpretable in general?

5.2.4 Convergence Behavior

Show how the convergence behavior is like for DRISPI by wallclock time and iterations. Use this chapter to also investigate improvement sources (quantitatively and qualitatively).

5.3 Limitations

Lay out the limitations of the study.

It should include the difficulty of comparing single-threaded monolithic solvers to the parallelized framework (CPU time vs. wallclock time, different hardware, etc.).

Also mention limitations and variations during the run when it comes to hardware and number of seeds. Computation was a limiting factor here and definitely influenced performance.

Also mention that DRISPI's online learning approach obviously benefits from longer runtimes, especially for larger instances (as you can see the low number of iterations there, and that HAOS weights haven't converged yet). This is also true for other instances, but especially the large and long-route ones would profit from HAOS convergence.

A further limitation concerns HAOS itself. Its contribution to the results above is established descriptively, through the weight convergence traced in Section 5.2.3, rather than against a fixed-configuration control arm: the same instances under the same wall-clock budget, with the HAOS draw frozen at a single configuration for the whole run, isolating what the adaptive draw adds over the one configuration a non-adaptive deployment would have to commit to in advance. That control arm was not run. Compute budget on the shared department server (Section 5.1.1) is why, not the design of this study: the 300-run campaign behind Section 5.2.1 already committed the server allocation available for this thesis, and a control arm needs a full campaign of its own run alongside it to be a controlled comparison rather than a confound, not a subset carved out of the existing one. This is not a new failure mode for this line of work. Altendörfer and Bachmann (2026) report it directly for DRSCI, a DRISPI predecessor for the CVRP built on the same decompose-route-improve principle: their results show a substantial share of DRSCI's own computation time spent evaluating arms an adaptive selection mechanism went on to discard as suboptimal, time a fixed-configuration run would not have spent at all. Whether HAOS incurs the same cost or improves on it is exactly the question a fixed-configuration control arm would answer, and the convergence account of Section 5.2.3 does not substitute for it.

6 Conclusion

Short introduction. It should relect on the primary and secondary research questions.

6.1 Highlights

Mention the strengths (such as adaptiveness through online learning) and weaknesses (such as more CPU time required) of the framework.

Also elaborate on the main contributions of the thesis.

6.2 Future Research

Use the HAOS weights to train an offline learning model and inject the results back into the framework. This effectively eliminates the warm-up period and will lead to a substantial speed-up of the framework.

Also mention that the framework, other than monolithic solvers, is highly adaptable to different instances and VRP variants. Since it only requires the plug-and-play of capable clustering methods and solvers on the their respective stages.

Bibliography

- Accorsi, L., D. Vigo. 2021. A fast and scalable heuristic for the solution of large-scale capacitated vehicle routing problems. *Transportation Science*, 55 (4), 832-856, doi: 10.1287/trsc.2021.1059.
- Accorsi, L., D. Vigo. 2024. Routing one million customers in a handful of minutes. *Computers & Operations Research*, 164 106562, doi: 10.1016/j.cor.2024.106562.
- Altendeitering, L., R. Bachmann. 2026. DRSCI: An Algorithmic Framework for Very Large-Scale Capacitated Vehicle Routing Problems. Ph.D. thesis.
- Auer, P., N. Cesa-Bianchi, P. Fischer. 2002. Finite-time analysis of the multiarmed bandit problem. *Machine Learning*, 47 (2-3), 235-256, doi: 10.1023/A:1013689704352.
- Balinski, M. L., R. E. Quandt. 1964. On an integer program for a delivery problem. *Operations Research*, 12 (2), 300-304, doi: 10.1287/opre.12.2.300.
- Barr, R. S., B. L. Golden, J. P. Kelly, M. G. C. Resende, W. R. Stewart. 1995. Designing and reporting on computational experiments with heuristic methods. *Journal of Heuristics*, 1 (1), 9-32, doi: 10.1007/BF02430363.
- Bezdek, J. C., R. Ehrlich, W. Full. 1984. FCM: The fuzzy c-means clustering algorithm. *Computers & Geosciences*, 10 (2-3), 191-203, doi: 10.1016/0098-3004(84)90020-7.
- Cavaliere, F., E. Bendotti, M. Fischetti. 2022. An integrated local-search/set-partitioning refinement heuristic for the Capacitated Vehicle Routing Problem. *Mathematical Programming Computation*, 14 (4), 749-779, doi: 10.1007/s12532-022-00224-2.
- Dekker, R., J. Bloemhof, I. Mallidis. 2012. Operations research for green logistics – an overview of aspects, issues, contributions and challenges. *European Journal of Operational Research*, 219 (3), 671-679, doi: 10.1016/j.ejor.2011.11.010.
- Demsar, J. 2006. Statistical Comparisons of Classifiers over Multiple Data Sets. *Journal of Machine Learning Research*, 7 (1), 1-30. URL <https://www.jmlr.org/papers/v7/demsar06a.html>.
- Fialho, Á., L. Da Costa, M. Schoenauer, M. Sebag. 2010. Analyzing bandit-based adaptive operator selection mechanisms. *Annals of Mathematics and Artificial Intelligence*, 60 (1-2), 25-64, doi: 10.1007/s10472-010-9213-y.

- Fisher, M. L., R. Jaikumar. 1981. A generalized assignment heuristic for vehicle routing. *Networks*, 11 (2), 109-124, doi: 10.1002/net.3230110205.
- Garfinkel, R. S., G. L. Nemhauser. 1969. The set-partitioning problem: Set covering with equality constraints. *Operations Research*, 17 (5), 848-856, doi: 10.1287/opre.17.5.848.
- Gillett, B. E., L. R. Miller. 1974. A heuristic algorithm for the vehicle-dispatch problem. *Operations Research*, 22 (2), 340-349, doi: 10.1287/opre.22.2.340.
- Jaccard, P. 1901. Distribution de la flore alpine dans le bassin des dranses et dans quelques régions voisines. *Bulletin de la Société Vaudoise des Sciences Naturelles*, 37 241-272, doi: 10.5169/seals-266440.
- Johnson, D. S. 2002. A theoretician's guide to the experimental analysis of algorithms. Michael H. Goldwasser, David S. Johnson, Catherine C. McGeoch, eds., *Data Structures, Near Neighbor Searches, and Methodology: Fifth and Sixth DIMACS Implementation Challenges, DIMACS Series in Discrete Mathematics and Theoretical Computer Science*, vol. 59. American Mathematical Society, Providence, RI, 215-250, doi: 10.1090/dimacs/059/11.
- Jourdan, L., M. Basseur, E.-G. Talbi. 2009. Hybridizing exact methods and metaheuristics: A taxonomy. *European Journal of Operational Research*, doi: 10.1016/J.EJOR.2007.07.035.
- Kaufman, L., P. J. Rousseeuw. 1990. *Finding Groups in Data: An Introduction to Cluster Analysis*. Wiley Series in Probability and Statistics, John Wiley & Sons, New York, doi: 10.1002/9780470316801.
- Kerscher, C., S. Minner. 2025. Decompose-route-improve framework for solving large-scale vehicle routing problems with time windows. *Transportation Research Part E: Logistics and Transportation Review*, 204 104409, doi: 10.1016/j.tre.2025.104409.
- Law, A. M., W. D. Kelton. 2000. *Simulation Modeling and Analysis*. 3rd ed. McGraw-Hill, Boston.
- Lenstra, J. K., A. H. G. R. Kan. 1981. Complexity of vehicle routing and scheduling problems. *Networks*, 11 (2), 221-227, doi: 10.1002/net.3230110211.
- Lourenço, H. R., O. C. Martin, T. Stützle. 2003. Iterated Local Search. Fred Glover, Gary A. Kochenberger, eds., *Handbook of Metaheuristics*. Springer US, Boston, MA, 320-353, doi: 10.1007/0-306-48056-5_11.
- Lübbecke, M. E., J. Desrosiers. 2005. Selected Topics in Column Generation. *Operations Research*, 53 (6), 1007-1023, doi: 10.1287/opre.1050.0234.
- MacQueen, J. B. 1967. Some methods for classification and analysis of multivariate observations. *Proceedings of the Fifth Berkeley Symposium on Mathematical Statistics and Probability*, vol. 1. University of California Press, Berkeley,

- CA, 281-297. URL <https://projecteuclid.org/euclid.bsmmsp/1200512992>.
- Martin, O., S. W. Otto, E. W. Felten. 1991. Large-step Markov chains for the Traveling Salesman Problem. *Complex Systems*, 5 (3), 299-326.
- Matl, P., R. F. Hartl, T. Vidal. 2018. Workload equity in vehicle routing problems: A survey and analysis. *Transportation Science*, 52 (2), 239-260, doi: 10.1287/trsc.2017.0744.
- Máximo, V. R., J.-F. Cordeau, M. C. V. Nascimento. 2024. AILS-II: An adaptive iterated local search heuristic for the large-scale capacitated vehicle routing problem. *INFORMS Journal on Computing*, 36 (4), 974-986, doi: 10.1287/ijoc.2023.0106.
- Máximo, V. R., M. C. V. Nascimento. 2021. A hybrid adaptive iterated local search with diversification control to the capacitated vehicle routing problem. *European Journal of Operational Research*, 294 (3), 1108-1119, doi: 10.1016/j.ejor.2021.02.024.
- Pisinger, D., S. Ropke. 2007. A general heuristic for vehicle routing problems. *Computers & Operations Research*, 34 (8), 2403-2435, doi: 10.1016/j.cor.2005.09.012.
- Queiroga, E., R. Martinelli, A. Subramanian, E. Uchoa, T. Vidal. 2026. The XL instances and the CVRPLib best known solution challenge. doi: 10.48550/arXiv.2601.11467. ArXiv:2601.11467.
- Rochat, Y., E. D. Taillard. 1995. Probabilistic diversification and intensification in local search for vehicle routing. *Journal of Heuristics*, 1 (1), 147-167, doi: 10.1007/BF02430370.
- Ropke, S., D. Pisinger. 2006. An Adaptive Large Neighborhood Search Heuristic for the Pickup and Delivery Problem with Time Windows. *Transportation Science*, 40 (4), 455-472, doi: 10.1287/trsc.1050.0135.
- Santini, A., M. Schneider, T. Vidal, D. Vigo. 2023. Decomposition Strategies for Vehicle Routing Heuristics. *INFORMS Journal on Computing*, 35 (3), 543-559, doi: 10.1287/ijoc.2023.1288.
- Uchoa, E., D. Pecin, A. Pessoa, M. Poggi, T. Vidal, A. Subramanian. 2017. New benchmark instances for the Capacitated Vehicle Routing Problem. *European Journal of Operational Research*, 257 (3), 845-858, doi: 10.1016/j.ejor.2016.08.012.
- Vidal, T. 2022. Hybrid genetic search for the CVRP: Open-source implementation and SWAP* neighborhood. *Computers & Operations Research*, 140 105643, doi: 10.1016/j.cor.2021.105643.
- Vidal, T., T. G. Crainic, M. Gendreau, N. Lahrichi, W. Rei. 2012. A Hybrid Genetic

Algorithm for Multidepot and Periodic Vehicle Routing Problems. *Operations Research*, 60 (3), 611-624, doi: 10.1287/opre.1120.1048.

Winston, W. L. 2007. *Operations research: Applications and algorithms*. 4th ed. Brooks/Cole Thomson Learning, Belmont 2007.

Wouda, N. A., L. Lan, W. Kool. 2024. PyVRP: A high-performance VRP solver package. *INFORMS Journal on Computing*, 36 (4), 943-955, doi: 10.1287/i-joc.2023.0055.

Zhang, J., T. van Woensel. 2023. Dynamic vehicle routing with random requests: A literature review. *International Journal of Production Economics*, 256 108751, doi: 10.1016/j.ijpe.2022.108751.

Appendix

1 Set Covering Formulation

Section 4.5 introduces set covering (SC) as the fallback DRISPI runs in place of set partitioning (SP) when the route pool does not yet cover every customer at least a minimum number of times. With R the route pool at the time of the solve, c_r the cost of route $r \in R$, C the customer set of the instance (Chapter 3), $C_r \subseteq C$ the customer set of route r , and $x_r \in \{0, 1\}$ selecting route r , SC relaxes the partition equality of SP (Equation 4.12) to the cover inequality of Garfinkel and Nemhauser (1969), who characterize set partitioning as set covering under an equality constraint:

$$\min \sum_{r \in R} c_r x_r \quad (1)$$

$$\text{s.t.} \quad \sum_{r \in R: i \in C_r} x_r \geq 1, \quad \forall i \in C \quad (2)$$

$$x_r \in \{0, 1\}, \quad \forall r \in R. \quad (3)$$

As with SP, DRISPI solves the LP relaxation of Equation 3, $x_r \in [0, 1]$, first; the resulting fractional weights update the pool's quality scores in the same way regardless of whether the solve ends up running SC or SP, and the binary re-solve is warm-started from that relaxation.

2 XL Instance Detailed Results

Table 6.1 reports the per-instance detail underlying the Section 5.2.1 comparison: instance characteristics, DRISPI’s best cost and gap to the current BKS for all 100 XL instances, alongside the initial-BKS and current-BKS reference costs.

Table 6.1: Detailed results for DRISPI on the 100 XL instances.

#	Name	Dep	Cust	Dem	Q	r	DRISPI		Initial BKS		BKS
							Cost	Gap (%)	Cost	Gap (%)	Cost
1	XL-n1048-k237	E	R	SL	128	4.7	380,838	+0.19	380,211	+0.03	380,107
2	XL-n1094-k157	C	C (6)	U	7	7.8	112,431	+0.00	112,431	+0.00	112,431
3	XL-n1141-k112	R	RC (3)	50-100	761	10.2	95,888	+0.26	95,727	+0.09	95,638
4	XL-n1188-k96	R	RC (3)	Q	782	12.4	104,600	+0.19	104,415	+0.01	104,402
5	XL-n1234-k55	E	R	1-10	126	22.7	96,848	+0.25	96,647	+0.05	96,603
6	XL-n1281-k29	C	C (5)	1-100	2267	45.5	31,161	+0.25	31,101	+0.06	31,083
7	XL-n1328-k19	R	RC (3)	5-10	542	72.6	38,261	+0.16	38,247	+0.12	38,201
8	XL-n1374-k278	C	R	Q	248	4.9	233,247	+0.13	233,049	+0.05	232,940
9	XL-n1421-k232	E	C (4)	1-100	309	6.1	385,182	+0.14	384,826	+0.05	384,646
10	XL-n1468-k151	E	R	50-100	726	9.7	250,530	+0.20	250,166	+0.06	250,028
11	XL-n1514-k106	C	RC (4)	5-10	107	14.2	92,533	+0.15	92,425	+0.03	92,398
12	XL-n1561-k75	R	C (4)	U	21	21.9	101,602	+0.06	101,549	+0.01	101,540
13	XL-n1608-k39	C	RC (3)	SL	337	42.1	48,067	+0.12	48,021	+0.02	48,009
14	XL-n1654-k11	E	R	1-10	845	155.4	36,420	+0.14	36,385	+0.04	36,370
15	XL-n1701-k562	R	C (6)	50-100	227	3.0	521,691	+0.16	521,136	+0.06	520,843
16	XL-n1748-k271	R	C (3)	Q	270	6.4	174,235	+0.27	173,896	+0.08	173,759
17	XL-n1794-k163	C	R	U	11	11.2	141,739	+0.01	141,729	+0.00	141,724
18	XL-n1841-k126	E	RC (2)	SL	186	14.6	214,184	+0.09	214,038	+0.03	213,981
19	XL-n1888-k82	E	R	5-10	173	23.1	143,864	+0.21	143,623	+0.05	143,558
20	XL-n1934-k46	C	C (6)	1-100	2166	42.2	53,080	+0.19	53,013	+0.06	52,980
21	XL-n1981-k13	R	RC (2)	1-10	832	153.5	32,597	+0.16	32,580	+0.10	32,546
22	XL-n2028-k617	C	C (6)	50-100	247	3.3	544,854	+0.13	544,403	+0.04	544,166
23	XL-n2074-k264	E	R	1-100	401	7.8	422,507	+0.30	421,627	+0.09	421,250
24	XL-n2121-k186	R	RC (5)	1-10	62	11.4	283,579	+0.17	283,211	+0.04	283,105
25	XL-n2168-k138	C	R	Q	800	15.7	127,763	+0.43	127,298	+0.07	127,210
26	XL-n2214-k131	R	C (5)	U	17	17.8	154,730	+0.05	154,676	+0.02	154,652
27	XL-n2261-k54	E	RC (5)	5-10	319	42.5	99,135	+0.31	98,907	+0.08	98,826
28	XL-n2307-k34	C	R	SL	479	69.6	47,980	+0.09	47,958	+0.05	47,936
29	XL-n2354-k631	E	C (4)	5-10	28	3.7	941,156	+0.06	940,825	+0.02	940,634
30	XL-n2401-k408	R	RC (4)	Q	303	5.9	464,406	+0.29	463,473	+0.08	463,084
31	XL-n2447-k290	C	RC (2)	SL	150	8.4	218,961	+0.15	218,706	+0.03	218,633
32	XL-n2494-k194	E	C (2)	1-100	661	12.9	361,553	+0.14	361,205	+0.04	361,057
33	XL-n2541-k121	R	R	U	21	21.7	146,609	+0.18	146,390	+0.03	146,345
34	XL-n2587-k66	C	R	50-100	2986	39.6	73,598	+0.47	73,394	+0.19	73,254
35	XL-n2634-k17	R	C (4)	1-10	898	162.6	31,946	+1.03	31,641	+0.07	31,619
36	XL-n2681-k540	E	RC (3)	1-100	251	5.0	800,308	+0.30	798,603	+0.09	797,906
37	XL-n2727-k546	C	RC (5)	U	5	5.3	431,165	+0.01	431,134	+0.01	431,108
38	XL-n2774-k286	E	C (5)	50-100	731	9.7	408,453	+0.21	407,847	+0.06	407,617
39	XL-n2821-k208	R	R	SL	179	13.5	217,020	+0.18	216,763	+0.07	216,622
40	XL-n2867-k120	R	C (4)	5-10	180	23.9	166,120	+0.12	165,990	+0.04	165,916
41	XL-n2914-k95	C	RC (3)	Q	1663	30.8	89,247	+0.39	88,990	+0.10	88,902
42	XL-n2961-k55	E	R	1-10	297	53.8	108,475	+0.49	108,084	+0.13	107,945
43	XL-n3007-k658	C	R	1-10	25	4.4	522,806	+0.13	522,319	+0.04	522,126
44	XL-n3054-k461	E	RC (4)	50-100	497	6.6	784,094	+0.24	782,739	+0.07	782,194
45	XL-n3101-k311	R	C (3)	SL	159	10.0	246,297	+0.22	245,937	+0.07	245,766
46	XL-n3147-k232	R	RC (6)	5-10	102	13.6	257,002	+0.23	256,626	+0.08	256,417
47	XL-n3194-k161	C	R	Q	1012	19.9	149,197	+0.40	148,728	+0.09	148,596
48	XL-n3241-k115	E	C (4)	1-100	1404	28.3	221,653	+0.18	221,370	+0.05	221,249
49	XL-n3287-k30	C	C (2)	U	111	112.0	40,315	+0.31	40,229	+0.10	40,190
50	XL-n3334-k934	E	R	1-10	20	3.5	1,453,320	+0.07	1,452,698	+0.02	1,452,353

Table 6.1 – continued from previous page

#	Name	Dep	Cust	Dem	Q	r	DRISPI		Initial BKS		BKS
							Cost	Gap (%)	Cost	Gap (%)	Cost
51	XL-n3408-k524	R	RC (3)	Q	353	6.5	679,883	+0.27	678,643	+0.09	678,024
52	XL-n3484-k436	E	C (6)	U	8	8.2	703,486	+0.02	703,355	+0.01	703,319
53	XL-n3561-k229	C	RC (5)	1-100	779	15.6	210,003	+0.39	209,386	+0.09	209,191
54	XL-n3640-k211	R	R	5-10	130	17.2	190,170	+0.34	189,724	+0.10	189,531
55	XL-n3721-k77	E	RC (2)	SL	371	48.8	163,281	+0.40	162,862	+0.14	162,630
56	XL-n3804-k29	C	R	50-100	10064	134.0	52,930	+0.17	52,885	+0.09	52,838
57	XL-n3888-k1010	R	C (2)	SL	128	4.2	1,881,512	+0.23	1,880,368	+0.17	1,877,119
58	XL-n3975-k687	C	RC (4)	1-10	32	5.6	526,648	+0.21	525,901	+0.06	525,560
59	XL-n4063-k347	E	R	Q	598	11.7	550,156	+0.33	548,931	+0.11	548,330
60	XL-n4153-k291	R	C (3)	1-100	726	14.3	356,629	+0.24	356,034	+0.07	355,780
61	XL-n4245-k203	R	R	U	21	21.0	229,919	+0.17	229,659	+0.06	229,521
62	XL-n4340-k148	E	RC (6)	50-100	2204	29.3	245,204	+0.54	244,226	+0.14	243,895
63	XL-n4436-k48	C	C (4)	5-10	706	94.1	61,636	+0.33	61,477	+0.07	61,431
64	XL-n4535-k1134	R	C (4)	U	4	4.3	1,203,553	+0.00	1,203,566	+0.00	1,203,512
65	XL-n4635-k790	C	RC (4)	1-100	294	5.9	612,069	+0.38	610,650	+0.15	609,734
66	XL-n4738-k487	E	R	Q	499	9.7	762,041	+0.31	760,501	+0.11	759,649
67	XL-n4844-k321	R	R	SL	188	15.1	405,321	+0.22	404,652	+0.06	404,429
68	XL-n4951-k203	E	RC (5)	50-100	1848	24.5	286,170	+0.49	285,269	+0.18	284,769
69	XL-n5061-k184	C	C (5)	5-10	206	27.4	161,968	+0.30	161,629	+0.09	161,480
70	XL-n5174-k55	C	C (6)	1-10	520	94.1	61,776	+0.85	61,382	+0.21	61,254
71	XL-n5288-k1246	E	R	50-100	318	4.2	1,961,708	+0.18	1,960,101	+0.10	1,958,205
72	XL-n5406-k783	R	RC (2)	1-10	38	6.8	1,041,209	+0.10	1,040,536	+0.04	1,040,141
73	XL-n5526-k553	R	C (3)	U	10	10.0	337,155	+0.09	336,898	+0.02	336,843
74	XL-n5649-k401	E	R	SL	181	14.1	645,503	+0.15	644,866	+0.05	644,562
75	XL-n5774-k290	C	RC (4)	1-100	1012	19.9	251,102	+0.48	250,207	+0.12	249,895
76	XL-n5902-k122	E	RC (3)	Q	2663	48.8	218,441	+0.66	217,447	+0.20	217,012
77	XL-n6034-k61	R	C (5)	5-10	744	98.9	64,945	+1.08	64,448	+0.31	64,251
78	XL-n6168-k1922	C	R	1-100	162	3.2	1,533,229	+0.38	1,530,010	+0.17	1,527,390
79	XL-n6305-k1042	R	RC (2)	Q	268	6.0	1,179,220	+0.27	1,177,528	+0.13	1,176,002
80	XL-n6445-k628	E	R	5-10	77	10.2	997,672	+0.16	996,623	+0.05	996,117
81	XL-n6588-k473	C	C (4)	1-10	76	13.8	334,751	+0.28	334,068	+0.07	333,822
82	XL-n6734-k330	R	RC (2)	50-100	1534	20.4	449,273	+0.40	448,031	+0.12	447,486
83	XL-n6884-k148	E	C (4)	SL	357	46.4	182,472	+0.49	181,809	+0.12	181,585
84	XL-n7037-k38	C	R	U	187	187.1	70,981	+0.31	70,845	+0.12	70,760
85	XL-n7193-k1683	E	RC (2)	5-10	32	4.2	2,960,620	+0.08	2,958,979	+0.03	2,958,205
86	XL-n7353-k1471	R	R	U	5	5.1	1,537,979	+0.02	1,537,811	+0.01	1,537,702
87	XL-n7516-k859	C	C (2)	1-100	439	8.8	575,245	+0.39	573,902	+0.16	572,984
88	XL-n7683-k602	R	RC (3)	50-100	957	12.8	703,448	+0.33	702,098	+0.14	701,151
89	XL-n7854-k365	E	C (2)	SL	223	21.5	659,690	+0.09	659,221	+0.02	659,075
90	XL-n8028-k294	C	R	Q	1386	27.3	267,910	+0.61	266,900	+0.23	266,295
91	XL-n8207-k108	C	C (2)	1-10	415	75.9	118,763	+0.60	118,274	+0.19	118,050
92	XL-n8389-k2028	E	RC (4)	1-100	208	4.1	3,365,000	+0.35	3,358,731	+0.16	3,353,305
93	XL-n8575-k1297	R	R	1-10	36	6.5	1,090,089	+0.16	1,089,137	+0.07	1,088,376
94	XL-n8766-k1032	R	RC (4)	50-100	637	8.5	908,402	+0.37	906,406	+0.15	905,060
95	XL-n8960-k634	E	C (5)	5-10	106	14.1	774,081	+0.15	773,383	+0.06	772,906
96	XL-n9160-k379	C	R	SL	237	24.2	325,066	+0.45	324,092	+0.15	323,596
97	XL-n9363-k209	C	RC (2)	U	45	45.2	206,063	+0.34	205,575	+0.10	205,369
98	XL-n9571-k55	R	R	Q	8773	174.7	108,022	+1.58	106,791	+0.42	106,344
99	XL-n9784-k2774	E	C (4)	1-10	19	3.4	4,078,780	+0.05	4,078,217	+0.03	4,076,936
100	XL-n10001-k1570	E	RC (2)	50-100	479	6.4	2,336,473	+0.25	2,333,757	+0.13	2,330,666

Declaration Of Authorship

I hereby solemnly declare that I have completed the present work independently. All ideas taken directly or indirectly from external sources have been clearly identified as such.

I am aware that the work may be examined in digital form to determine whether unauthorized aids were used and whether it constitutes - either in whole or in part - plagiarism. For the purpose of comparison with existing sources, my work may be entered into a database and, after review, may remain there for comparison with future submissions. No further reproduction or usage rights are thereby granted. This work has neither been submitted to another examination authority nor published.

Place, Date

Lucca Altendeitering

Declaration of AI Use

Download the Declaration of AI Use from the website, complete it truthfully, and include it as a separate page at the end of your final thesis PDF here instead of this page.